Introduction à C++ et Rcpp

Hervé Perdry

2019-03-13

Table des matières

1 À propos de ce document **5**

2 Objets de R **7**
2.1 Types . . . 7
2.2 Attributs . . . 8
2.3 Mieux examiner les objets . . . 9
2.4 Pour les apprentis sorciers . . . 10

3 Bases de C++ **13**
3.1 Nécessité des types . . . 13
3.2 Les entiers . . . 14
3.3 Les flottants . . . 15
3.4 Constantes numériques . . . 17
3.5 Opérateurs arithmétiques . . . 17
3.6 Booléens . . . 18
3.7 Tableaux de taille fixe . . . 19
3.8 Contrôle du flux d'exécution : les boucles . . . 19
3.9 Contrôle du flux d'exécution : les alternatives . . . 22
3.10 Premiers objets Rcpp : les vecteurs . . . 23

4 Manipuler les objets de R **27**
4.1 Objets génériques : SEXP . . . 27
4.2 Vecteurs . . . 27
4.3 Vecteurs nommés . . . 30
4.4 Sucre syntaxique . . . 32
4.5 Facteurs . . . 35
4.6 Listes et Data frames . . . 36

5 Exemple : Metropolis-Hastings **39**
5.1 L'algorithme . . . 39
5.2 Une version R . . . 39
5.3 Première version en C++ . . . 39
5.4 Comparaisons . . . 40
5.5 Une version améliorée . . . 40

6 Un peu plus de C++ **43**
6.1 Pointeurs . . . 43
6.2 Références . . . 45
6.3 Les objets de Rcpp sont passés par référence . . . 47
6.4 Surcharge . . . 48
6.5 Templates . . . 49
6.6 C++11 . . . 51

7 Exemple : Metropolis-Hastings (2) 55
7.1 Une version templatée . . . 55
7.2 Mise en œuvre . . . 56

8 Chaînes de caractères 57
8.1 Chaînes de la vieille école : `C strings` . . . 58
8.2 Chaînes C++ . . . 60
8.3 Chaines `Rcpp::String` . . . 62
8.4 Exemple : lecture d'un fichier . . . 62
8.5 Addendum sur les listes . . . 63

9 Exemple : chaînes de caractère et spécialisation de template 65

10 Matrices 67
10.1 Le *column major mode* . . . 67
10.2 Matrices avec `RcppArmadillo` . . . 68
10.3 Matrices avec `RcppEigen` . . . 70
10.4 Benchmarking (1) . . . 72
10.5 Benchmarking sur l'algorithme Nipals . . . 72
10.6 Conclusion ? . . . 74

11 Exemple : la régression logistique 75
11.1 Modèle et algorithme . . . 75
11.2 Code Armadillo . . . 75
11.3 Code Eigen . . . 76
11.4 Benchmarking . . . 77

12 Introduction à la librairie standard 79
12.1 Vecteurs . . . 79
12.2 Quelques algorithmes . . . 80
12.3 Pairs & tuples . . . 80
12.4 tuples . . . 81
12.5 Maps . . . 81
12.6 Heap . . . 81

13 Programmation objet 83
13.1 Classes . . . 83
13.2 Hérédité . . . 89
13.3 Fonctions virtuelle et classes abstraites . . . 89
13.4 Copie, Move, Destruction . . . 90

14 Exemple : Metropolis-Hastings (3) 93
14.1 Une classe fonctionnelle . . . 93
14.2 Mise en œuvre . . . 93

15 Multi-threading avec RcppParallel 95
15.1 Classes `RVector` et `RMatrix` . . . 95
15.2 Classe `Worker` . . . 96
15.3 Utilisation de `ParallelReduce` . . . 96
15.4 Utilisation de `ParallelFor` . . . 98
15.5 Un exemple contre-productif avec `ParallelFor` . . . 99

16 Créer un package 101

Chapitre 1

À propos de ce document

Ce document est issu d'un cours d'école doctorale donné en février/mars 2019. Un effort a été fait pour le rendre lisible sans les commentaires fait en cours, mais il y a certainement toujours des lacunes.

Les exemples de code proposés utilisent souvent le standard C++11. Pour pouvoir compiler avec ce standard, utilisez `Sys.setenv("PKG_CXXFLAGS" = "-std=c++11")`.

Les comparaisons de temps d'exécution qui apparaissent ici ont été obtenues avec une installation de R « standard » (pas de librairie comme openBlas ou autre), une compilation avec `clang++`, sur une machine linux disposant de 8 cœurs à 3.60 GHz, avec un cache de 8 MB. Des comparaisons avec d'autres compilateurs ou sur d'autres machines peuvent donner des résultats (très) différents, tant en valeur des temps d'exécution qu'en comparaison des performances.

L'idéal serait d'amener les lecteurs d'une part à une bonne connaissance des possibilités offertes par `Rcpp`, d'autre part au niveau nécessaire pour ouvrir `The C++ programming language` de Bjarne Stroustrup – on recommande, avant de se frotter à cet énorme et patibulaire ouvrage de référence (1300 pages), le plus court et plus amène `A tour of C++` du même auteur.

Chapitre 2

Objets de R

On supposera que les objets de R sont bien connus. Dans ce court chapitre nous allons simplement voir comment examiner leur structure.

2.1 Types

L'instruction `typeof` permet de voir le type des objets. Considérons trois vecteurs, une matrice, une liste, un data frame, un facteur.

```
> a <- c("bonjour", "au revoir")
> b <- c(TRUE, FALSE, TRUE, TRUE)
> c <- c(1.234, 12.34, 123.4, 1234)
> d <- matrix( rpois(12, 2), 4, 3)
> e <- list(un = a, deux = b)
> f <- data.frame(b, c)
> g <- factor( c("F", "M", "F", "F") )
> typeof(a) # chaines de caractères
[1] "character"
```

```
> typeof(b) # valeurs vrai / faux
[1] "logical"
```

```
> typeof(c) # nombres "à virgule"
[1] "double"
```

```
> typeof(d) # nombres entiers
[1] "integer"
```

```
> typeof(e) # liste
[1] "list"
```

```
> typeof(f) # data frame = liste
[1] "list"
```

```
> typeof(g) # facteur = entiers (!)
[1] "integer"
```

Il y a deux types de variables numériques : `double` (nombres « à virgule », en formant dit « flottant ») et `integer` (entiers). Les entiers s'obtiennent en tapant `0L`, `1L`, etc; certaines commandes renvoient des entiers:

```
> typeof(0)
[1] "double"
```

```
> typeof(0L)
[1] "integer"
```

```
> typeof(0:10)
[1] "integer"
```

```
> typeof(which(c > 100))
[1] "integer"
```

```
> typeof(rpois(10, 1))
[1] "integer"
```

On remarque que le facteur g a pour type `integer`. Ce petit mystère s'éclaircira bientôt.

Pour examiner le contenu d'un objet avec une information sur son type, on peut utiliser `str`.

```
> str(a) # chaines de caractères
 chr [1:2] "bonjour" "au revoir"
```

```
> str(b) # vrai / faux
 logi [1:4] TRUE FALSE TRUE TRUE
```

```
> str(c) # "à virgule"
 num [1:4] 1.23 12.34 123.4 1234
```

```
> str(d) # entiers (matrice 4x3)
 int [1:4, 1:3] 2 0 2 5 0 2 1 1 1 1 ...
```

```
> str(e) # liste
List of 2
 $ un  : chr [1:2] "bonjour" "au revoir"
 $ deux: logi [1:4] TRUE FALSE TRUE TRUE
```

```
> str(f) # data frame
'data.frame':   4 obs. of  2 variables:
 $ b: logi  TRUE FALSE TRUE TRUE
 $ c: num  1.23 12.34 123.4 1234
```

```
> str(g) # facteur
 Factor w/ 2 levels "F","M": 1 2 1 1
```

2.2 Attributs

Les objets de R ont des « attributs ». Ainsi donner des noms aux éléments de `c` revient à lui donner un attribut `names`

```
> names(c) <- c("elt1", "elt2", "elt3", "elt4")
> c
    elt1     elt2     elt3     elt4
   1.234   12.340  123.400 1234.000
```

```
> attributes(c)
$names
```

```
[1] "elt1" "elt2" "elt3" "elt4"
```

Ce qui différentie une matrice d'un vecteur, c'est l'attribut `dim`:

```
> attributes(d)
$dim
[1] 4 3
```

Les data frames et les facteurs ont également des attributs :

```
> attributes(f)
$names
[1] "b" "c"

$class
[1] "data.frame"

$row.names
[1] 1 2 3 4
```

```
> attributes(g)
$levels
[1] "F" "M"

$class
[1] "factor"
```

Les attributs peuvent être modifiés avec la syntaxe `attributes(x) <- ...` ou un individuellement avec `attr(x, which)` :

```
> attr(d, "dim")
[1] 4 3
```

```
> attr(d, "dim") <- c(2L, 6L)
> d
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    2    2    0    1    1    5
[2,]    0    5    2    1    1    1
```

2.3 Mieux examiner les objets

La fonction `dput` permet d'obtenir une forme qui peut être copiée dans une autre session R ; ceci permet parfois d'obtenir des informations plus précises sur la représentation interne d'un objet. Nous allons l'utiliser ici pour mieux comprendre la construction des matrices, des data frames, et des facteurs.

Il est nécessaire de jeter au préalable un œil à l'aide de `structure` pour mieux comprendre le résultat. On y précise notamment :

```
For historical reasons (these names are used when deparsing),
attributes ‘".Dim"’, ‘".Dimnames"’, ‘".Names"’, ‘".Tsp"’ and
‘".Label"’ are renamed to ‘"dim"’, ‘"dimnames"’, ‘"names"’,
‘"tsp"’ and ‘"levels"’.
```

2.3.1 Cas de la matrice

```
> dput(d)
structure(c(2L, 0L, 2L, 5L, 0L, 2L, 1L, 1L, 1L, 1L, 5L, 1L), .Dim = c(2L,
6L))
```

Une matrice est un vecteur muni d'un attribut `dim` (qui apparaît comme `.Dim` dans le résultat de `dput`).

2.3.2 Cas du data frame :

```
> dput(f)
structure(list(b = c(TRUE, FALSE, TRUE, TRUE), c = c(1.234, 12.34,
123.4, 1234)), class = "data.frame", row.names = c(NA, -4L))
```

Un data frame est une liste munie d'un attribut `class = "data.frame"` et d'un attribut `row.names` (ici, la valeur de cet attribut est la convention pour « 4 lignes non nommées »).

2.3.3 Cas du facteur :

```
> dput(g)
structure(c(1L, 2L, 1L, 1L), .Label = c("F", "M"), class = "factor")
```

```
> levels(g)
[1] "F" "M"
```

Un facteur est qu'un vecteur d'entiers muni d'attributs `class = "factor"`. et `levels` (les niveaux du facteur), qui apparaît dans `structure` sous le nom `.Label` ; cet attribut est également accessible via la fonction `levels`.

On peut par exemple fabriquer un facteur à partir d'un vecteur d'entiers, ainsi :

```
> h <- c(2L, 1L, 1L, 2L)
> attributes(h) <- list(levels = c("L1", "L2"), class = "factor")
> h
[1] L2 L1 L1 L2
Levels: L1 L2
```

2.4 Pour les apprentis sorciers

La fonction « interne » `inspect` permet de voir l'adresse où se trouve l'objet, son type (d'abord codé numériquement, par exemple 13 pour `integer` puis le nom conventionnel de ce type, `INTSXP`), et quelques autres informations ; les objets complexes (leurs attributs) sont « déroulés ».

```
> .Internal(inspect( 1:10 ))
@55f0d0ba3968 13 INTSXP g0c0 [NAM(3)]  1 : 10 (compact)
```

```
> .Internal(inspect( c(0.1, 0.2) ))
@55f0cf8f3e98 14 REALSXP g0c2 [] (len=2, tl=0) 0.1,0.2
```

```
> a <- c(0.1, 0.2)
> .Internal(inspect( a ))
@55f0cf8bb258 14 REALSXP g0c2 [NAM(3)] (len=2, tl=0) 0.1,0.2
```

```
> names(a) <- c("A", "B")
> .Internal(inspect( a ))
@55f0cf81dff8 14 REALSXP g0c2 [NAM(1),ATT] (len=2, tl=0) 0.1,0.2
ATTRIB:
  @55f0ccee4ff8 02 LISTSXP g0c0 []
    TAG: @55f0c3872a60 01 SYMSXP g1c0 [MARK,NAM(3),LCK,gp=0x4000] "names" (has value)
    @55f0cf81df78 16 STRSXP g0c2 [NAM(3)] (len=2, tl=0)
      @55f0c5f4cd70 09 CHARSXP g1c1 [MARK,gp=0x61] [ASCII] [cached] "A"
      @55f0c6f11398 09 CHARSXP g1c1 [MARK,gp=0x61] [ASCII] [cached] "B"
```

```
> .Internal(inspect( h ))
@55f0cb05d738 13 INTSXP g0c2 [OBJ,NAM(3),ATT] (len=4, tl=0) 2,1,1,2
ATTRIB:
  @55f0cee8d198 02 LISTSXP g0c0 []
    TAG: @55f0c3872bb0 01 SYMSXP g1c0 [MARK,NAM(3),LCK,gp=0x6000] "levels" (has value)
    @55f0cb05d638 16 STRSXP g0c2 [NAM(3)] (len=2, tl=0)
      @55f0c6c74c70 09 CHARSXP g1c1 [MARK,gp=0x61] [ASCII] [cached] "L1"
      @55f0c6c74bc8 09 CHARSXP g1c1 [MARK,gp=0x60] [ASCII] [cached] "L2"
    TAG: @55f0c3872f30 01 SYMSXP g1c0 [MARK,NAM(3),LCK,gp=0x6000] "class" (has value)
    @55f0ccb70768 16 STRSXP g0c1 [NAM(3)] (len=1, tl=0)
      @55f0c391bff8 09 CHARSXP g1c1 [MARK,gp=0x61] [ASCII] [cached] "factor"
```

Les plus braves pourront consulter le code de cette fonction, ainsi que tout le code de R, à cette adresse : https://github.com/wch/r-source/tree/trunk/src plus précisément pour `inspect`, dans `src/main/inspect.c`...

Chapitre 3

Bases de C++

Il faut toujours commencer par saluer le monde. Créez un fichier `hello.cpp` contenant le code suivant :

```
#include <Rcpp.h>
#include <iostream>
//[[Rcpp::export]]
void hello() {
  Rcpp::Rcout << "Hello world!\n";
}
```

Compilez le depuis R (il faut avoir installé le package Rcpp) :

```
library(Rcpp)
sourceCpp("hello.cpp")
```

(ou, si vous utilisez R Studio, cliquez sur « source »...). Appelez ensuite la fonction en R :

```
> hello()
Hello world!
```

Dans le programme C++, les directives d'inclusion `#include` servent à inclure des librairies. La librairie `Rcpp.h` permet l'interaction avec les objets de R ; la définition de l'objet `Rcpp::Rcout`, un « flux de sortie » (output stream) qui permet l'écriture dans la console R y est incluse. La librairie `iostream` contient en particulier la définition de l'opérateur `<<`. Elle n'est en fait pas nécessaire ici car `Rcpp.h` contient une directive d'inclusion similaire.

3.1 Nécessité des types

C++ est un langage compilé et non interprété. Le compilateur est le programme qui lit le code C++ et produit un code assembleur puis du langage machine (possiblement en passant par un langage intermédiaire).

Les instructions de l'assembleur (et du langage machine qui est sa traduction numérique directe) manipulent directement les données sous formes de nombre codés en binaire, sur 8, 16, 32 ou 64 bits. La manipulation de données complexes (des vecteurs, des chaines de caractères) se fait bien sûr en manipulant une suite de tels nombres.

Pour que le compilateur puisse produire de l'assembleur, il faut qu'il sache la façon dont les données sont codées dans les variables. La conséquence est que toutes les variables doivent déclarées, et ne pourrons pas changer de type ; de même, le type des valeurs retournées par les fonctions doit être fixé, ainsi que celui de leurs paramètres.

Les fantaisies permises par R (voir ci-dessous) ne sont plus possibles (étaient-elles souhaitables ?).

```
fantaisies <- function(a) {
  if(a == 0) {
    return(a)
  } else {
    return("Non nul")
  }
}
```

```
> fantaisies(0)
[1] 0
```

```
> fantaisies(1)
[1] "Non nul"
```

```
> fantaisies("0")
[1] "0"
```

```
> fantaisies("00")
[1] "Non nul"
```

La librairie standard de C++ offre une collection de types de données très élaborés et de fonctions qui les manipulent. Nous commencerons par les types fondamentaux : entiers, flottants, booléens.

3.2 Les entiers

Compilez ce programme qui affiche la taille (en octets) des quatre types d'entiers (signés) (le résultat peut théoriquement varier d'une architecture à l'autre, c'est-à-dire qu'il n'est pas fixé par la description officielle du C++).

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
void int_types() {
  char a;
  short b;
  int c;
  long int d;
  Rcout << "sizeof(a) = " << sizeof(a) << "\n";
  Rcout << "sizeof(b) = " << sizeof(b) << "\n";
  Rcout << "sizeof(c) = " << sizeof(c) << "\n";
  Rcout << "sizeof(d) = " << sizeof(d) << "\n";
}
```

Notez une nouveauté ci-dessus : la directive `using namespace Rcpp` qui permet de taper `Rcout` au lieu de `Rcpp::Rcout`. C'est commode mais à utiliser avec parsimonie (et à bon escient) : il n'est en effet pas rare que des fonctions appartenant à des namespace différents portent le même nom. La syntaxe `namespace::fonction` permet d'éviter toute ambiguïté.

```
> int_types()
sizeof(a) = 1
sizeof(b) = 2
sizeof(c) = 4
sizeof(d) = 8
```

Notez le `using namespace Rcpp` qui permet de taper simplement `Rcout` au lieu de `Rcpp::Rcout`.

Les entiers de R correspondent au type `int`(sur 32 bits) mais cela ne vous empêche pas de manipuler dans vos fonctions C++ des entiers plus courts ou plus longs si vous en avez besoin.

Il existe aussi des types non signés, par exemple `unsigned int` ou `unsigned char` ; et des raccourcis variés, par exemple `size_t` pour `unsigned long int`.

3.2.0.1 Notre première fonction « non void »

Écrivons notre première fonction qui renvoie une valeur. Son type doit être déclaré comme ceci :

```
//[[Rcpp::export]]
int somme_entiers(int a, int b) {
  return a+b;
}
```

Et testons la :

```
> somme_entiers(1L, 101L)
[1] 102
```

```
> somme_entiers(1.9,3.6)
[1] 4
```

Que se passe-t-il ? Utilisez la fonction suivante pour comprendre.

```
//[[Rcpp::export]]
int cast_to_int(int x) {
  return x;
}
```

3.2.1 Initialisation des variables

Il est nécessaire d'initialiser les variables.

```
//[[Rcpp::export]]
int uninit() {
  int a; // a peut contenir n'importe quoi
  return a;
}
```

Testons :

```
> uninit()
[1] 1879123923
```

On aura parfois 0, mais pas systématiquement (cela dépend de l'état de la mémoire). On peut initialiser `a` lors de la déclaration : `int a = 0;`

3.3 Les flottants

Il y a trois types de nombres en format flottant. Le type utilisé par R est le `double` de C++.

```
#include <Rcpp.h>
//[[Rcpp::export]]
void float_types() {
  float a;
```

```
  double b;
  long double c;
  Rcpp::Rcout << "sizeof(a) = " << sizeof(a) << "\n";
  Rcpp::Rcout << "sizeof(b) = " << sizeof(b) << "\n";
  Rcpp::Rcout << "sizeof(c) = " << sizeof(c) << "\n";
}
```

3.3.1 Précision du calcul

Parenthèse de culture informatique générale. Voici ce que répond R au test `1.1 - 0.9 = 0.2`.

```
> 1.1 - 0.9 == 0.2
[1] FALSE
```

Pourquoi ? Est-ce que C++ fait mieux ? (Rappel : R utilise des `double`).

Sur les architectures courantes, les nombres au format `double` sont codés sur 64 bits (voir ci-dessus, taille 8 octets). C'est un format « à virgule flottante », c'est-à-dire qu'ils sont représentés sous la forme $a2^b$, ou a et b sont bien sûr codés en binaire (sur 53 bits – dont un bit 1 implicite – pour a, 11 pour b, et un bit de signe). Cette précision finie implique des erreurs d'arrondi. Pour plus de détails, voir Wikipedia sur la norme IEEE 754 : https://fr.wikipedia.org/wiki/IEEE_754

Quelle est la différence entre les nombres ci-dessus ?

```
> (1.1 - 0.9) - 0.2
[1] 5.551115e-17
```

C'est-à-dire 2^{-54} (une erreur au 53e chiffre...). Affichons la représentation interne des nombres en question avec la fonction `bits` du package `pryr`.

```
> pryr::bits(1.1 - 0.9)
[1] "00111111 11001001 10011001 10011001 10011001 10011001 10011001 10011100"
```

```
> pryr::bits(0.2)
[1] "00111111 11001001 10011001 10011001 10011001 10011001 10011001 10011010"
```

3.3.2 Valeurs spéciales et extrêmes

Il y a des valeurs spéciales en C++ comme en R : une valeur infinie, et une valeur non-définie `NaN`, pour *not a number*.

```
#include <Rcpp.h>
//[[Rcpp::export]]
double divise(double a, double b) {
  double r = a/b;
  Rcpp::Rcout << a << " / " << b << " = " << r << std::endl;
  return r;
}
```

```
> divise(1,2)
1 / 2 = 0.5
[1] 0.5
```

```
> divise(1,0)
1 / 0 = inf
[1] Inf
```

```
> divise(-1,0)
-1 / 0 = -inf
[1] -Inf
```

```
> divise(0,0)
0 / 0 = -nan
[1] NaN
```

En C++, la fonction `numeric_limits` permet d'obtenir les valeurs extrêmes que peuvent prendre les `double`.

```
#include <Rcpp.h>
//[[Rcpp::export]]
void numeric_limits() {
  Rcpp::Rcout
    << "plus petite valeur positive 'normale' = "
    << std::numeric_limits<double>::min()         << "\n"
    << "plus petite valeur positive           = "
    << std::numeric_limits<double>::denorm_min()  << "\n"
    << "plus grande valeur                    = "
    << std::numeric_limits<double>::max() << "\n"
    << "epsilon                               = "
    << std::numeric_limits<double>::epsilon()     << "\n";
}
```

```
> numeric_limits()
plus petite valeur positive 'normale' = 2.22507e-308
plus petite valeur positive           = 4.94066e-324
plus grande valeur                    = 1.79769e+308
epsilon                               = 2.22045e-16
```

3.4 Constantes numériques

Attention, si le R considère que `0` ou `1` est un `double` (il faut taper `0L` ou `1L` pour avoir un `integer`), pour C++ ces valeurs sont des entiers. Pour initialiser proprement un `double` il faut taper `0.` ou `0.0`, etc. La plupart du temps le compilateur corrige ces petites erreurs.

3.5 Opérateurs arithmétiques

Les opérateurs arithmétiques sont bien entendu `+`, `-`, `*` et `/`. Pour les entiers, le modulo est `%`.

```
#include <Rcpp.h>
// [[Rcpp::export]]
void division_entiere(int a, int b) {
  int q = a / b;
  int r = a % b;
  Rcpp::Rcout << a << " = " << b << " * " << q << " + " << r << std::endl;
}
```

```
> division_entiere(128, 7)
128 = 7 * 18 + 2
```

À ces opérateurs, il faut ajouter des opérateurs d'assignation composée `+=`, `-=`, `*=` et `/=` qui fonctionnent ainsi : `x += 4;` est équivalent à `x = x + 4`, et ainsi de suite. Il y a aussi les opérateurs d'incrémentation `++` et de

décrémentation `--`.

```
#include <Rcpp.h>
// [[Rcpp::export]]
void operateurs_exotiques(int a) {
  Rcpp::Rcout << "a = " << a << std::endl;

  Rcpp::Rcout << "a *= 2;" << std::endl;
  a *= 2;
  Rcpp::Rcout << "a = " << a << std::endl;

  Rcpp::Rcout << "int b = a++;" << std::endl;
  int b = a++; // post incrementation
  Rcpp::Rcout << "b = " << b << std::endl;
  Rcpp::Rcout << "a = " << a << std::endl;

  Rcpp::Rcout << "int c = ++a;" << std::endl;
  int c = ++a; // pre incrementation
  Rcpp::Rcout << "c = " << c << std::endl;
  Rcpp::Rcout << "a = " << a << std::endl;
}
```

```
> operateurs_exotiques(3)
a = 3
a *= 2;
a = 6
int b = a++;
b = 6
a = 7
int c = ++a;
c = 8
a = 8
```

3.6 Booléens

Le type `bool` peut prendre les valeurs vrai/faux. Il correspond au type `logical` de R.

```
// [[Rcpp::export]]
bool test_positif(double x) {
  return (x > 0);
}
```

Les opérateurs de test sont comme en R, `>`, `>=`, `<`, `<=`, `==` et `!=`. Les opérateurs logiques sont `&&` (et), `||` (ou) et `!` (non). **Attention !** Les opérateurs `&` et `|` existent également, ce sont des opérateurs logiques bit à bit qui opèrent sur les entiers.

```
// [[Rcpp::export]]
bool test_interval(double x, double min, double max) {
  return (min <= x && x <= max);
}
```

3.7 Tableaux de taille fixe

On peut définir des tableaux de taille fixe fixe (connue à la compilation) ainsi:

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
//[[Rcpp::export]]
void petit_tableau() {
  int a[4] = {0,2,7,11};
  SHOW(a)    // wut ?
  SHOW(a[0])
  SHOW(a[1])
  SHOW(a[2])
  SHOW(a[3])
}
```

L'occasion est saisie pour montrer l'utilisation d'une macro. La ligne `SHOW(a[0])` est remplacée par `Rcpp::Rcout << "a[0]" << " = " << (a[0]) << std::endl;` **avant** la compilation. Les macros peuvent rendre de grand services pour la clarté du code ou pour faciliter le débuggage « manuel ».

L'utilisation de parenthèse autour de `(x)` dans la définition de la macro est très conseillée : si on utilisait par exemple `SHOW(a == b)` il n'y a aucun problème avec la syntaxe `Rcout << (a == b) << std::endl;` mais `Rcout << a == b << std::endl;` pourrait poser des problèmes de priorité des opérateurs `==` et `<<`...

Le résultat de `SHOW(a)` sera expliqué plus tard (pointeurs).

3.8 Contrôle du flux d'exécution : les boucles

3.8.1 Boucles `for`

Plus de 90% des boucles `for` s'écrivent ainsi :

```
#include <Rcpp.h>
// [[Rcpp::export]]
void ze_loop(int n) {
  for(int i = 0; i < n; i++) {
    Rcpp::Rcout << "i = " << i << std::endl;
  }
}
```

```
> ze_loop(4)
i = 0
i = 1
i = 2
i = 3
```

Le premier élément dans la parenthèse (ici, `int i = 0`) est l'initialisation ; il sera exécuté une seule fois, et c'est généralement une déclaration de variable (avec une valeur initiale). Le deuxième élément (`i < n`) est la condition à laquelle la boucle sera exécutée une nouvelle fois, c'est généralement une condition sur la valeur de la variable ; et le dernier élément (`i++`) est exécuté à la fin de chaque tour de boucle, c'est généralement une mise à jour de la valeur de cette variable.

Il est facile par exemple d'aller de 2 en 2 :

```
#include <Rcpp.h>
// [[Rcpp::export]]
```

```
void bouclette(int n) {
  for(int i = 0; i < n; i += 2) {
    Rcpp::Rcout << "i = " << i << std::endl;
  }
}
```

```
> bouclette(6)
i = 0
i = 2
i = 4
```

Pour revenir sur les types d'entiers : gare au dépassement arithmétique.

```
#include <Rcpp.h>
// [[Rcpp::export]]
void arithmetic_overflow() {
  int x = 1;
  for(int i = 0; i < 33; i++) {
    Rcpp::Rcout << "2^" << i << " = " << (x) << "\n";
    x = 2*x;
  }
}
```

Essayer avec `unsigned int`, `long int`.

3.8.2 `continue` et `break`

Une instruction `continue` en cours de boucle fait passer au tour suivant :

```
#include <Rcpp.h>
// [[Rcpp::export]]
void trois(int n) {
  for(int i = 0; i < n; i++) {
    Rcpp::Rcout << i ;
    if(i%3 == 0)
      continue;
    Rcpp::Rcout << ".";
  }
  Rcpp::Rcout << std::endl;
}
```

```
> trois(12)
01.2.34.5.67.8.910.11.
```

Quant à `break`, si s'agit bien sûr d'une interruption de la boucle.

```
#include <Rcpp.h>
// [[Rcpp::export]]
void zz(int n, int z) {
  for(int i = 0; i < n; i++) {
    Rcpp::Rcout << "A" ;
    if(i > z)
      break;
  }
  Rcpp::Rcout << std::endl;
}
```

```
> zz(14, 100)
AAAAAAAAAAAAAA
```

```
> zz(14, 5)
AAAAAAA
```

3.8.3 Boucles `while` et `do while`

Ces boucles ressemblent fort à ce qui existe en R. Dans un cas, le test est fait avant la boucle, dans l'autre il est fait après.

```
#include <Rcpp.h>
// [[Rcpp::export]]
void a_rebours_1(int n) {
  while(n-- > 0) {
    Rcpp::Rcout << n << " ";
  }
  Rcpp::Rcout << std::endl;
}

// [[Rcpp::export]]
void a_rebours_2(int n) {
  do {
    Rcpp::Rcout << n << " ";
  } while(n-- > 0);
  Rcpp::Rcout << std::endl;
}
```

```
> a_rebours_1(3)
2 1 0
```

```
> a_rebours_2(3)
3 2 1 0
```

On peut aussi utiliser `continue` et `break` dans ces boucles.

Considérons un exemple un peu moins artificiel : le calcul d'une racine carrée par l'algorithme de Newton. L'avantage de la syntaxe `do while` est apparent ici.

```
#include <Rcpp.h>
// [[Rcpp::export]]
double racine_carree(double x, double eps = 1e-5) {
  double s = x;
  do {
    s = 0.5*(s + x/s);
  } while( fabs(s*s - x) > eps);
  return s;
}
```

```
> racine_carree(2)
[1] 1.414216
```

```
> racine_carree(2, 1e-8)
[1] 1.414214
```

Cherchez sur le site `cpprefrence.com` la description des fonctions `abs` et `fabs`. Pourquoi ne pouvait-on pas utiliser `abs` ici ? Est-il raisonnable de proposer une valeur trop petite pour `eps` ? Proposer une modification de la

fonction qui évite cet écueil.

3.9 Contrôle du flux d'exécution : les alternatives

3.9.1 `if` et `if else`

Cela fonctionne tout à fait comme en R ; la x

```
// [[Rcpp::export]]
double mini(double x, double y) {
  double re = 0;
  if(x > y) {
    re = y;
  } else {
    re = x;
  }
  return re;
}
```

```
> mini(22, 355)
[1] 22
```

3.9.2 `switch`

Un exemple simple devrait permettre de comprendre le fonctionnement de `switch`.

```
#include <Rcpp.h>
// [[Rcpp::export]]
void combien(int n) {
  switch(n) {
    case 0:
      Rcpp::Rcout << "aucun\n";
      break;
    case 1:
      Rcpp::Rcout << "un seul\n";
      break;
    case 2:
      Rcpp::Rcout << "deux\n";
      break;
    case 3:
    case 4:
    case 5:
      Rcpp::Rcout << "quelques uns\n";
      break;
    default:
      Rcpp::Rcout << "beaucoup\n";
  }
}
```

3.10 Premiers objets Rcpp : les vecteurs

La librairie `Rcpp` définit des types `NumericVector`, `IntegerVector` et `LogicalVector` qui permettent de manipuler en C++ les vecteurs de R.

3.10.1 Créer des vecteurs, les manipuler

L'initialisation avec la syntaxe utilisée dans `vec0` remplit le vecteur de 0. Notez l'accès aux éléments d'un vecteur par l'opérateur `[]`; **contrairement à la convention utilisée par R, les vecteurs sont numérotés de 0 à n-1.**

```
#include <Rcpp.h>
// Création d'un vecteur (initialisé à 0)
// [[Rcpp::export]]
Rcpp::NumericVector vec0(int n) {
  Rcpp::NumericVector x(n);
  return x;
}

// accès aux éléments
// [[Rcpp::export]]
Rcpp::IntegerVector vec1(int n) {
  Rcpp::IntegerVector x(n);
  for(int i = 0; i < n; i++) {
    x[i] = i*i;
  }
  return x;
}
```

3.10.2 Exemple : compter les zéros

```
#include <Rcpp.h>
//[[Rcpp::export]]
int count_zeros(Rcpp::IntegerVector x) {
  int re = 0;
  // x.size() et x.length() renvoient la taille de x
  int n = x.size();
  for(int i = 0; i < n; i++) {
    re += (x[i] == 0)?1:0;
  }
  return re;
}
```

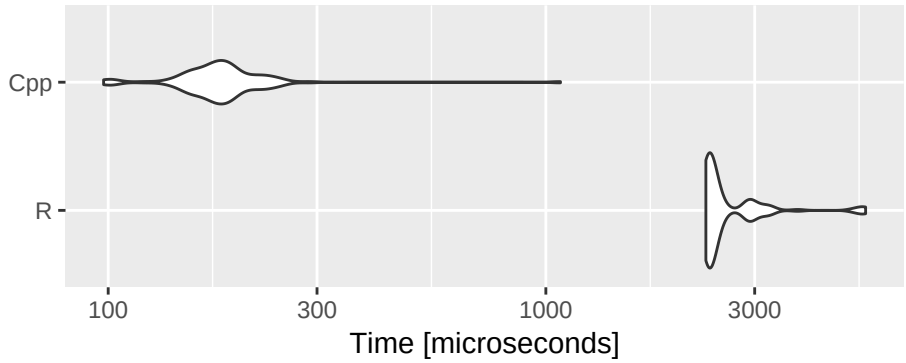
On a utilisé ici l'*opérateur ternaire* `test?a:b`, qui vaut `a` si `test` est `true`, `b` sinon. Comment les performances de cette fonction se comparent-elles avec le code R `sum(a == 0)` ?

```
> set.seed(1); a <- sample(0:99, 1e6, TRUE)
> count_zeros(a);
[1] 9816
```

```
> sum(a == 0)
[1] 9816
```

```
> mbm <- microbenchmark::microbenchmark( R = sum(a == 0), Cpp = count_zeros(a))
> mbm
Unit: microseconds
 expr      min        lq      mean   median       uq      max neval
    R 2320.865 2337.0050 2659.7802 2404.851 2738.967 5377.943   100
  Cpp   97.617  160.7145  189.0121  178.224  194.112 1079.815   100
```

```
> ggplot2::autoplot(mbm)
```



La différence de vitesse d'exécution s'explique en partie par le fait que le code R commence par créer un vecteur de type `logical` (le résultat de `a == 0`), puis le parcourt pour faire la somme. Ceci implique beaucoup de lectures écritures en mémoire, ce qui ralentit l'exécution.

La solution suivante a été proposée en cours.

```
#include <Rcpp.h>
//[[Rcpp::export]]
int count_zeros2(Rcpp::IntegerVector x) {
  int re = 0;
  int n = x.size();
  for(int i = 0; i < n; i++) {
    (x[i] == 0)?re++:0;
  }
  return re;
}
```

Mais en fait, comme en R, ajouter un booléen à un int produit une conversion `false` : 0 et `true` : 1.

```
#include <Rcpp.h>
//[[Rcpp::export]]
int count_zeros3(Rcpp::IntegerVector x) {
  int re = 0;
  int n = x.size();
  for(int i = 0; i < n; i++) {
    re += (x[i] == 0);
  }
  return re;
}
```

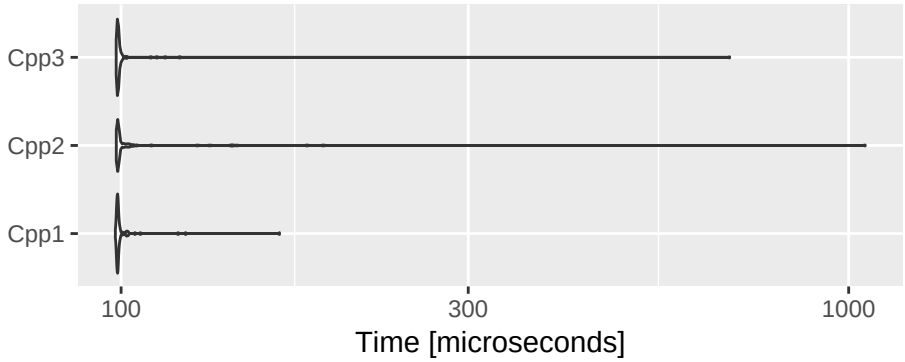
Une comparaison s'impose.

```
> count_zeros2(a)
[1] 9816
```

```
> count_zeros3(a)
[1] 9816
```

```
> mbm <- microbenchmark::microbenchmark( Cpp1 = count_zeros(a),
+             Cpp2 = count_zeros2(a),
+             Cpp3 = count_zeros3(a) )
> mbm
Unit: microseconds
 expr    min      lq     mean  median      uq      max neval
 Cpp1 98.203 98.7175 100.3931 98.9705 99.3365  165.086   100
 Cpp2 98.495 98.7775 112.7609 99.0750 99.8805 1053.801   100
 Cpp3 98.464 98.7860 105.5877 98.9970 99.3295  686.126   100
```

```
> ggplot2::autoplot(mbm)
```



Selon les options de compilation ou les architectures, il peut y avoir ou non une différence entre ces solutions.

Chapitre 4

Manipuler les objets de R

4.1 Objets génériques : SEXP

Les objets R les plus génériques sont les `SEXP`, « S expression ». Les principaux types de `SEXP` sont illustrés par la fonction suivante.

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
std::string le_type(SEXP x) {
  switch( TYPEOF(x) ) {
    case INTSXP:
      return "integer";
    case REALSXP:
      return "double";
    case LGLSXP:
      return "logical";
    case STRSXP:
      return "character";
    case VECSXP:
      return "list";
    case NILSXP:
      return "NULL";
    default:
      return "autre";
  }
}
```

Ça n'est généralement pas une bonne idée d'écrire des fonctions génériques comme celle-ci. Utiliser les types définis par Rcpp est généralement plus facile et plus sûr. Cependant nous verrons plus loin qu'à l'intérieur des fonctions Rcpp ils peuvent être utiles, par exemple dans le cas où une fonction renvoie soit un `NILSXP`, soit un objet d'un autre type.

4.2 Vecteurs

On a vu l'initialisation avec la syntaxe `NumericVector R(n)` qui crée un vecteur de longueur n, rempli de 0. On peut utiliser `NumericVector R(n, 1.0)` pour un vecteur rempli de 1 ; **attention à bien taper** `1.0` **pour avoir**

un `double` et non un `int`; dans le cas contraire, on a un message d'erreur difficilement compréhensible à la compilation.

On peut utiliser `NumericVector R = no_init(n);` pour un vecteur non initialisé (ce qui fait gagner du temps d'exécution).

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
IntegerVector zeros(int n) {
  IntegerVector R(n);
  return R;
}
//[[Rcpp::export]]
IntegerVector whatever(int n, int a) {
  IntegerVector R(n, a);
  return R;
}
//[[Rcpp::export]]
IntegerVector uninitialized(int n) {
  IntegerVector R = no_init(n);
  return R;
}
//[[Rcpp::export]]
IntegerVector nombres_fetiches() {
  IntegerVector R = IntegerVector::create(1, 4, 8);
  return R;
}
```

```
> zeros(5)
[1] 0 0 0 0 0
```

```
> whatever(5, 2L)
[1] 2 2 2 2 2
```

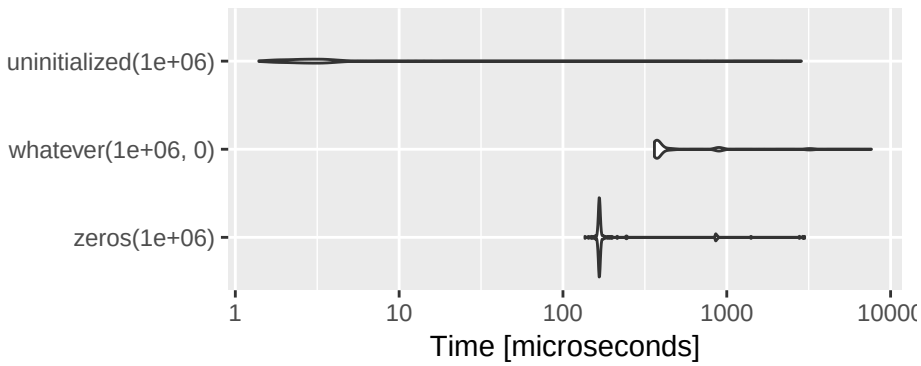
```
> uninitialized(5) # sometime 0s, not always
[1] -802688408      22000 -802688464      22000 -769402904
```

```
> nombres_fetiches()
[1] 1 4 8
```

Comparons les performances des trois premières fonctions (comme à chaque fois, les résultats peuvent varier d'une architecture à l'autre).

```
> mbm <- microbenchmark::microbenchmark(zeros(1e6), whatever(1e6, 0), uninitialized(1e6))
> mbm
Unit: microseconds
                 expr     min       lq      mean   median      uq      max neval
         zeros(1e+06) 136.232 165.7130 305.15403 167.3705 169.799 2988.237   100
   whatever(1e+06, 0) 361.819 371.4165 711.34515 377.2415 457.527 7640.235   100
 uninitialized(1e+06)   1.392   2.1490  95.39591   2.7855   3.521 2854.872   100
```

```
> ggplot2::autoplot(mbm)
```



4.2.1 Gestions des valeurs manquantes

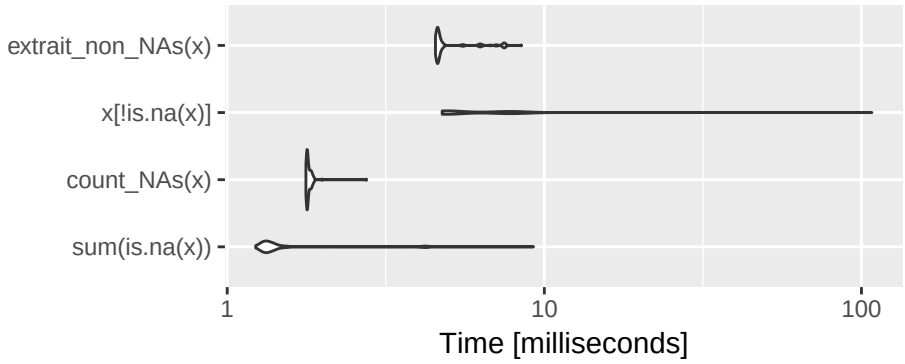
Cette fonction utilise `IntegerVector::is_na` qui est la bonne manière de tester si un membre d'un vecteur entier est `NA`.

```
#include <Rcpp.h>
using namespace Rcpp;
// [[Rcpp::export]]
int count_NAs(NumericVector x) {
  int re = 0;
  int n = x.size();
  for(int i = 0; i < n; i++) {
    re += NumericVector::is_na(x[i]);
  }
  return(re);
}
// [[Rcpp::export]]
NumericVector extrait_non_NAs(NumericVector x) {
  int nb_nas = count_NAs(x);
  int n = x.size();
  NumericVector R(n - nb_nas);
  int j = 0;
  for(int i = 0; i < n; i++) {
    if(!NumericVector::is_na(x[i])) {
      R[j++] = x[i];
    }
  }
  return R;
}
```

Comparons ces deux fonctions avec leurs analogues R, `sum(is.na(x))` et `x[!is.na(x)]`.

```
> x <- sample( c(NA, rnorm(10)), 1e6, TRUE)
> mbm <- microbenchmark::microbenchmark( sum(is.na(x)), count_NAs(x), x[!is.na(x)], extrait_non_NAs(x)
> mbm
Unit: milliseconds
               expr      min       lq     mean   median       uq        max neval
      sum(is.na(x)) 1.229974 1.304990 1.678301 1.346649 1.425860   9.239834   100
       count_NAs(x) 1.770566 1.783089 1.814983 1.794104 1.825678   2.749513   100
       x[!is.na(x)] 4.760144 4.879087 7.081225 5.026361 7.715680 107.871419   100
 extrait_non_NAs(x) 4.535547 4.595811 5.117628 4.634502 4.782358   8.474959   100
```

```
> ggplot2::autoplot(mbm)
```



4.3 Vecteurs nommés

Ça n'est pas passionnant en soi (on ne manipule pas si souvent des vecteurs nommés), mais ce qu'on voit là sera utile pour les listes et les data frames.

4.3.1 Créer des vecteurs nommés

Voici d'abord comment créer un vecteur nommé.

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
NumericVector creer1() {
  NumericVector x = NumericVector::create(Named("un") = 10, Named("deux") = 20);
  return x;
}
```

Application :

```
> a <- creer1()
> a
  un deux
  10   20
```

Une syntaxe plus dense est possible :

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
NumericVector creer2() {
  NumericVector x = NumericVector::create(_["un"] = 10, _["deux"] = 20);
 return x;
}
```

Cela produit le même résultat.

```
> creer2()
  un deux
  10   20
```

4.3.2 Obtenir les noms d'un vecteur

Et voici comment obtenir les noms d'un vecteur.

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
CharacterVector noms(NumericVector x) {
  CharacterVector R = x.names(); // ou R = x.attr("names");
  return R;
}
```

Utilisons cette fonction:

```
> noms(a)
[1] "un"   "deux"
```

```
> b <- seq(0,1,length=6)
> noms(b)
Error in noms(b): Not compatible with STRSXP: [type=NULL].
```

Bien sûr, le vecteur `b` n'a pas de noms ; la fonction `x.names()` a renvoyé l'objet `NULL`, de type `NILSXP`, qui ne peut être utilisé pour initialiser le vecteur `R` de type `STRSXP`. La solution est d'attraper le résultat de `x.names()` dans un `SEXP`, et de tester son type avec `TYPEOF`.

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
CharacterVector noms2(NumericVector x) {
  SEXP R = x.names();
  if( TYPEOF(R) == STRSXP )
    return R;
  else
    return CharacterVector(0);
}
```

```
> noms2(a)
[1] "un"   "deux"
```

```
> noms2(b)
character(0)
```

4.3.3 Accéder aux éléments par leurs noms

On utilise toujours la syntaxe `x[]` :

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
double get_un(NumericVector x) {
  if(x.containsElementNamed("un"))
    return x["un"];
  else
    stop("Pas d'élément 'un'");
}
```

Notez la fonction `Rcpp::stop` qui correspond à la fonction R du même nom.

```
> get_un(a)
[1] 10
```

```
> get_un(b)
Error in get_un(b): Pas d'élément 'un'
```

4.3.4 Énumerer les noms et le contenu

On va utiliser l'opérateur `CHAR` qui, appliqué à un élément d'un `CharacterVector`, renvoie une valeur de type `const char *` c'est-à-dire un pointeur vers une chaîne de caractère (constante, ie non modifiable) « à la C » (voir chapitre dédié).

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
void enumere(NumericVector x) {
  SEXP r0 = x.names();
  if(TYPEOF(r0) != STRSXP) {
    Rcout << "Pas de noms\n";
    return;
  }
  CharacterVector R(r0);
  for(int i = 0; i < R.size(); i++) {
    double a = x[ CHAR(R[i]) ];
    Rcout << CHAR(R[i]) << " : " << a << "\n";
  }
}
```

```
> enumere(a)
un : 10
deux : 20
```

4.4 Sucre syntaxique

La fonction R suivante fait un usage abondant de la vectorisation.

```
bonne.vectorisation <- function(x, y) {
  z <- 3*x + y
  if(any(x > 1))
    z <- z*2;
  sum( ifelse(z > 0, z, y) )
}
```

```
> set.seed(1); x <- rnorm(10); y <- rnorm(10)
> bonne.vectorisation(x,y)
[1] 26.99719
```

La transcription en C++ devrait faire intevenir trois boucles ; c'est un peu fastidieux. Le sucre syntaxique ajouté par les fonctions dites *Rcpp sugar* permet d'éviter de les écrire.

```
#include <Rcpp.h>
using namespace Rcpp;
// [[Rcpp::export]]
double fonction_sucree(NumericVector x, NumericVector y) {
```

```
  NumericVector z = 3*x + y;
  if( is_true(any(x > 1)) )
    z = z*2;
  return sum(ifelse(z > 0, z, y));
}
```

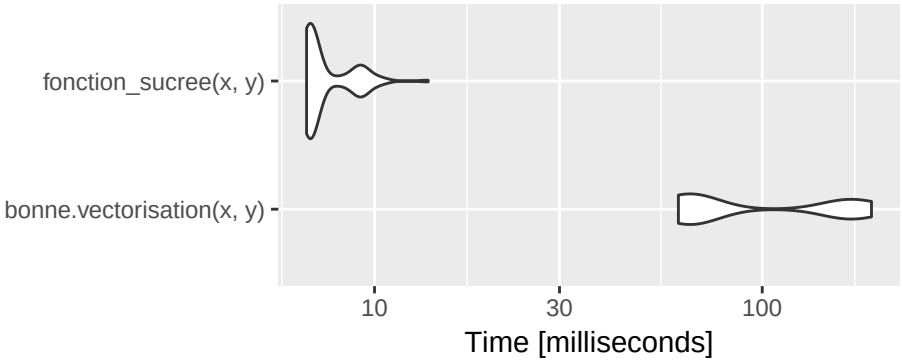
```
> fonction_sucree(x,y)
[1] 26.99719
```

4.4.1 Efficacité

Grâce à une implémentation soignée, les fonctions Rcpp sugar sont redoutablement efficaces.

```
> x <- rnorm(1e6)
> y <- rnorm(1e6)
> mbm <- microbenchmark::microbenchmark( bonne.vectorisation(x,y), fonction_sucree(x,y) )
> mbm
Unit: milliseconds
                      expr       min        lq       mean    median         uq       max
 bonne.vectorisation(x, y) 60.779426 64.165569 106.094586 67.292589 166.749223 191.33908
     fonction_sucree(x, y)  6.680222  6.803631   7.561056  6.865287   8.259474  13.75639
 neval
   100
   100
```

```
> ggplot2::autoplot(mbm)
```



Revenons cependant à notre exemple de comptage de 0.

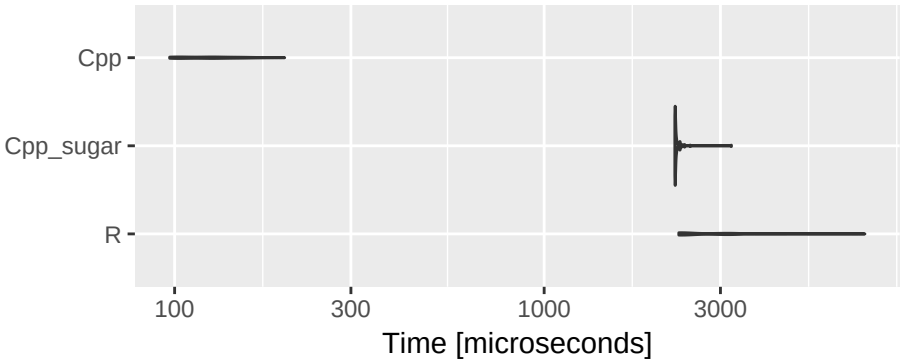
```
#include <Rcpp.h>
using namespace Rcpp;
// [[Rcpp::export]]
int count_zeros_sugar(IntegerVector x) {
  return sum(x == 0);
}
```

```
> set.seed(1); a <- sample(0:99, 1e6, TRUE)
> count_zeros_sugar(a);
[1] 9816
```

```
> mbm <- microbenchmark::microbenchmark( R = sum(a == 0), Cpp_sugar = count_zeros_sugar(a), Cpp = count_
> mbm
Unit: microseconds
```

```
      expr      min        lq      mean    median       uq      max neval
         R 2314.309 2331.0715 2784.4656 2396.5505 3166.358 7382.624   100
 Cpp_sugar 2258.041 2261.2995 2290.2370 2266.1220 2279.444 3212.139   100
       Cpp   96.946  104.8115  126.0593  123.9865  140.390  198.547   100
```

```
> ggplot2::autoplot(mbm)
```



On le voit, la fonction qui utilise le sucre syntaxique, bien que toujours très efficace, n'atteint pas toujours la performance d'une fonction plus rustique.

4.4.2 Un exemple de la Rcpp Gallery

Un exemple tiré de la Rcpp Gallery http://gallery.rcpp.org/articles/simulating-pi/ (estimation de π par la méthode de Monte-Carlo) et la variante avec une boucle.

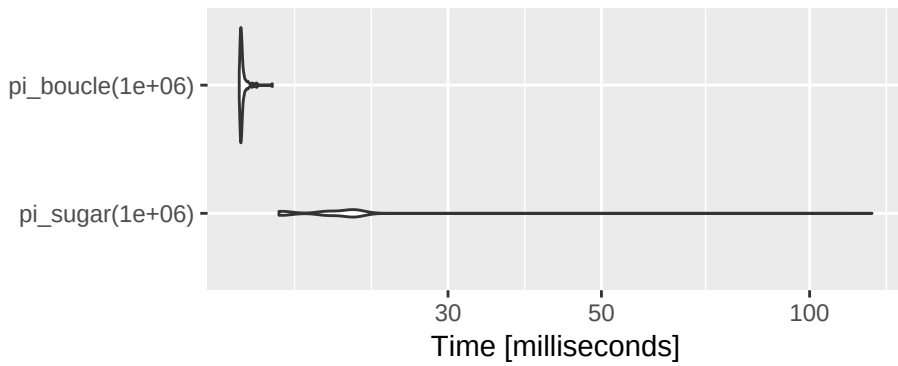
```
#include <Rcpp.h>
using namespace Rcpp;
// [[Rcpp::export]]
double pi_sugar(const int N) {
    NumericVector x = runif(N);
    NumericVector y = runif(N);
    // NumericVector d = sqrt(x*x + y*y);
    NumericVector d = x*x + y*y;
    return 4.0 * sum(d < 1.0) / N;
}

// [[Rcpp::export]]
double pi_boucle(const int N) {
  int S = 0;
  for(int i = 0; i < N; i++) {
    double x = R::runif(0, 1);
    double y = R::runif(0, 1);
    S += (x*x + y*y) < 1.0;
  }
  return (4.0 * (double) S / (double) N);
}
```

```
> pi_sugar(1e6)
[1] 3.142932
```

```
> pi_boucle(1e6)
[1] 3.142476
```

```
> mbm <- microbenchmark::microbenchmark(pi_sugar(1e6), pi_boucle(1e6))
> mbm
Unit: milliseconds
             expr      min       lq     mean   median       uq       max neval
  pi_sugar(1e+06) 17.08824 18.90879 21.39293 20.91421 21.94969 123.12032   100
 pi_boucle(1e+06) 14.97044 15.04007 15.15205 15.08268 15.15187  16.71471   100
```

```
> ggplot2::autoplot(mbm)
```



4.5 Facteurs

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
CharacterVector niveaux(IntegerVector x) {
  SEXP R = x.attr("levels");
  switch(TYPEOF(R)) {
  case STRSXP:
    return R; // Rcpp prend soin que ce SEXP soit converti en CharacterVector
  case NILSXP:
    stop("Pas d'attribut 'levels'");
  default:
    stop("Attribut 'levels' de type inconvenant");
  }
}
```

```
> x <- factor( sample(c("M","F"), 10, TRUE) )
> niveaux(x)
[1] "F" "M"
```

```
> x <- sample(1:2, 10, TRUE)
> # niveaux(x)
> attr(x, "levels") <- c(0.1, 0.4)
> # niveaux(x)
```

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
IntegerVector un_facteur() {
  IntegerVector x = IntegerVector::create(1,1,2,1);
  x.attr("levels") = CharacterVector::create("F", "M");
```

```
  x.attr("class") = CharacterVector::create("factor");
  return x;
}
```

```
> un_facteur()
[1] F F M F
Levels: F M
```

4.6 Listes et Data frames

Nous avons déjà vu les fonctions utiles dans le cas des vecteurs nommés, en particulier `containsElementNamed`.

La fonction suivante prend une liste `L` qui a un élément `L$alpha` de type `NumericVector` et renvoie celui-ci à l'utilisateur. En cas de problème un message d'erreur informatif est émis.

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
NumericVector get_alpha_list(List x) {
  if( x.containsElementNamed("alpha") ) {
    SEXP R = x["alpha"];
    if( TYPEOF(R) != REALSXP )
      stop("elt alpha n'est pas un 'NumericVector'");
    return R;
  } else
    stop("Pas d'elt alpha");
}
```

On peut modifier « en place » les éléments d'une liste:

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
void increment_alpha(List x) {
  if( x.containsElementNamed("alpha") ) {
    SEXP R = x["alpha"];
    if( TYPEOF(R) != REALSXP )
      stop("elt alpha n'est pas un 'NumericVector'");
    NumericVector Alpha(R);
    Alpha = Alpha+1;  // sugar
  } else
    stop("Pas d'elt alpha");
}
```

Exemple:

```
> x <- list( alpha = c(0.1,7), beta = 1:4)
> x
$alpha
[1] 0.1 7.0

$beta
[1] 1 2 3 4
```

```
> increment_alpha(x)
> x
$alpha
[1] 1.1 8.0

$beta
[1] 1 2 3 4
```

Pour renvoyer des valeurs hétéroclites dans une liste c'est très facile:

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
List cree_liste() {
  List L;
  L["a"] = NumericVector::create(1.0, 2.0, 4.0);
  L["b"] = 12;
  L["c"] = rnorm(4, 0.0, 1.0);
  return L;
}
```

Les data frames, ont l'a vu, sont des listes avec quelques attributs supplémentaires. En Rcpp cela fonctionne de la même façon, avec la classe `DataFrame`. Ils ont une certaine tendance à se transformer en liste quand on leur ajoute des éléments : il suffit la plupart de transformer le résultat en liste (avec `as.data.frame`) une fois qu'on l'a obtenu.

Chapitre 5

Exemple : Metropolis-Hastings

5.1 L'algorithme

TBC

5.2 Une version R

```
PI <- function(x) (1 + x[1]**2 + x[1]*x[2] + x[2]**2)^(-3)
MH <- function(B, sd) {
  R <- matrix(0.0, nrow = B, ncol = 2)
  x <- R[1,]
  for(b in 2:B) {
    y <- x + rnorm(2, 0, sd = sd)
    rho <- PI(y) / PI(x)
    u <- runif(1)
    if(u < rho)
      x <- y
    R[b, ] <- x
  }
  return(R);
}
```

5.3 Première version en C++

Une version obtenue en « traduisant » l'implémentation en R.

```
#include <Rcpp.h>
using namespace Rcpp;

double Pi(double x1, double x2) {
  return pow((1 + x1*x1 + x1*x2 + x2*x2),-3.0);
}
//[[Rcpp::export]]
NumericMatrix MH_cpp(int B, double sd) {
```

```
  NumericMatrix R(B, 2);
  double x1 = 0.0, x2 = 0.0;
  for(int b = 1; b < B; b++) {
    double y1 = x1 + R::rnorm(0,sd);
    double y2 = x2 + R::rnorm(0,sd);
    double rho = Pi(y1, y2) / Pi(x1, x2);
    double u = R::unif_rand();
    if(u < rho) {
      x1 = y1;
      x2 = y2;
    }
    R(b,0) = x1;
    R(b,1) = x2;
  }
  return R;
}
```

5.4 Comparaisons

TBC

5.5 Une version améliorée

Il est souhaitable d'éviter de recalculer `Pi(x1, x2)` à chaque tour de boucle, alors que cette valeur est déjà connue.

```
#include <Rcpp.h>
using namespace Rcpp;

double Pi(double x1, double x2) {
  return pow((1 + x1*x1 + x1*x2 + x2*x2),-3.0);
}
//[[Rcpp::export]]
NumericMatrix MH_cpp_1(int B, double sd, int burn = 0, int thin = 1) {
  NumericMatrix R(B, 2);
  thin = (thin < 1)?1:thin;
  double x1 = 0.0, x2 = 0.0;
  int b = 0;
  double pi_x = Pi(x1, x2);
  for(int k = 0; b < B ; k++) { // !! boucle exotique !!
    double y1 = x1 + R::rnorm(0,sd);
    double y2 = x2 + R::rnorm(0,sd);
    double pi_y = Pi(y1, y2) / Pi(x1, x2);
    double u = R::unif_rand();
    if(u * pi_x < pi_y) {
      x1 = y1;
      x2 = y2;
      pi_x = pi_y;
    }
    if(k > burn && (k % thin) == 0) {
      R(b,0) = x1;
```

```
      R(b,1) = x2;
      b++;
    }
    if((k % 1000) == 0) // toutes les mille itérations
      checkUserInterrupt();
  }
  return R;
}
```

Le paramètre `burn` permet de ne pas retenir les premières itérations ; le paramètre `thin` permet de ne retenir qu'une itération sur `thin` (pour réduire la dépendance entre les tirages successifs).

À noter : la fonction `checkUserInterrupt()` qui permet d'interrompre le programme en cas d'appui sur ctrl + C, ou sur le petit panneau STOP de R studio. On n'appelle pas cette fonction à chaque tour de boucle car elle est longue à calculer !

À noter également : une boucle exotique, puisque la condition d'arrêt n'est pas sur le compteur de boucle `k`, mais sur `b`, qui est régulièrement incrémenté dans la boucle (mais, en général, pas à chaque tour).

Chapitre 6

Un peu plus de C++

6.1 Pointeurs

Les pointeurs sont un héritage de C. Il s'agit de « pointer » vers l'adresse d'un objet ou une variable. Un pointeur vers un `int` est un `int *`. Si `p` est un pointeur, `*p` est l'objet à l'adresse pointée par `p`.

Pour obtenir le pointeur vers `x`, on utilise `&x`.

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
// [[Rcpp::export]]
void pointeurs() {
  int x = 12;   // entier
  int * p;      // pointeur vers un entier (non initialisé !!)
  p = &x;       // p pointe vers x
  SHOW(x);
  SHOW(p);
  SHOW(*p);
  Rcpp::Rcout << "On ajoute 1 à x\n";
  x += 1;
  SHOW(x);
  SHOW(p);
  Rcpp::Rcout << "On ajoute 1 à *p\n";
  *p += 1;
  SHOW(x);
  SHOW(p);
}
```

```
> pointeurs()
x = 12
p = 0x7ffcd4e5cc1c
*p = 12
On ajoute 1 à x
x = 13
p = 0x7ffcd4e5cc1c
On ajoute 1 à *p
x = 14
p = 0x7ffcd4e5cc1c
```

Nous avions déjà rencontré des pointeurs sans le savoir : les tableaux sont des pointeurs ! Plus précisément, si on a déclaré `int a[4]`, `a` pointe vers le premier élément du tableau, `a+1` vers le second, etc.

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
// [[Rcpp::export]]
void tableaux() {
  int a[4] = {10,20,30,40};
  SHOW(a);
  SHOW(a[0]);
  SHOW(*a);
  SHOW(a[2]);
  SHOW(a+2);
  SHOW(*a+2);
  SHOW(*(a+2));
}
```

```
> tableaux()
a = 0x7ffcd4e5cc00
a[0] = 10
*a = 10
a[2] = 30
a+2 = 0x7ffcd4e5cc08
*a+2 = 12
*(a+2) = 30
```

Notez la différence entre `a` et `a+2` : la taille des `int` est prise en compte dans le calcul.

En passant à une fonction un pointeur vers une variable, d'une part on permet la modification de cette variable, d'autre part on évite la copie de cette variable (intéressant quand on passe des objets de grande taille).

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;

// les arguments sont des pointeurs vers des entiers
inline void swap(int * x, int * y) {
  int tmp = *x;
  *x = *y;
  *y = tmp;
}

// [[Rcpp::export]]
void demonstrate_swap(int a, int b) {
  if(a > b) swap(&a,&b); // &a = pointeur vers a ...
  SHOW(a);
  SHOW(b);
}
```

Attention, la fonction `swap` ne peut pas être exportée vers R : les objets de R ne peuvent être transformés en `int *`, en pointeurs vers des entiers. Elle est destinée à n'être utilisée que dans notre code C++.

Le mot clef `inline` ci-dessus indique au compilateur qu'on désire que la fonction soit insérée dans le code aux endroit où elle est utilisée. Cela économise un appel de fonction…

```
> demonstrate_swap(12, 14)
a = 12
b = 14
```

```
> demonstrate_swap(14, 12)
a = 12
b = 14
```

6.2 Références

Le mécanisme derrière les références est similaire à celui des pointeurs ; la syntaxe est plus simple et permet d'éviter de promener des étoiles dans tout le code. Une référence à un entier est de type `int &`, et peut être manipulée directement comme un `int`. On l'initialise à l'aide d'une autre variable de type `int`.

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
// [[Rcpp::export]]
void references() {
  int x = 12;   // entier
  int & y = x;  // référence à x
  SHOW(x);
  SHOW(y);
  SHOW(&x);
  SHOW(&y);
  Rcpp::Rcout << "On ajoute 1 à x\n";
  x += 1;
  SHOW(x);
  SHOW(y);
  Rcpp::Rcout << "On ajoute 1 à y\n";
  y += 1;
  SHOW(x);
  SHOW(y);
}
```

```
> references()
x = 12
y = 12
&x = 0x7ffcd4e5cc1c
&y = 0x7ffcd4e5cc1c
On ajoute 1 à x
x = 13
y = 13
On ajoute 1 à y
x = 14
y = 14
```

C'est surtout utile pour spécifier les arguments d'une fonction. Voici ce que devient notre fonction `swap`:

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;

// les arguments sont des références à des entiers
inline void swap2(int & x, int & y) {
  int tmp = x;
  x = y;
  y = tmp;
}
```

```
// [[Rcpp::export]]
void demonstrate_swap2(int a, int b) {
  if(a > b) swap2(a,b); // on passe directement x et y
  SHOW(a);
  SHOW(b);
}
```

Cette fonction `swap2` ne peut pas plus être exportée que la fonction `swap` définie plus haut.

```
> demonstrate_swap(12, 14)
a = 12
b = 14
```

```
> demonstrate_swap(14, 12)
a = 12
b = 14
```

6.2.1 Autres exemples

Un appel `divise_par_deux(x)` est équivalent à `x /= 2`. L'utilisation de cette fonction ne simplifie pas vraiment le code, mais ce mécanisme pourrait être utile pour des opérations plus complexes:

```
void divise_par_deux(int & x) {
  x /= 2;
}

// [[Rcpp::export]]
int intlog2(int a) {
  int k = 0;
  while(a != 0) {
    divise_par_deux(a);
    k++;
  }
  return k-1;
}
```

```
> intlog2(17)
[1] 4
```

Une autre utilité est de renvoyer plusieurs valeurs. Bien sûr quand on utilise `Rcpp` on peut manipuler des vecteurs ou des listes, mais utiliser des variables passées par référence pour récupérer les résultats d'un calcul est souvent très commode.

```
#include <Rcpp.h>
#include <cmath>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;

bool eq_quadratique(double a, double b, double c, double & x1, double & x2) {
  double delta = b*b - 4*a*c;
  if(delta < 0)
    return false;
  double sqrt_delta = std::sqrt(delta);
  x1 = (-b - sqrt_delta)/(2*a);
  x2 = (-b + sqrt_delta)/(2*a);
  return true;
```

```
}
//[[Rcpp::export]]
void deuxieme_degre(double a, double b, double c) {
  double x1, x2;
  bool solvable = eq_quadratique(a, b, c, x1, x2);
  if(solvable) {
    SHOW(x1);
    SHOW(x2);
  } else {
    Rcpp::Rcout << "Pas de solution\n";
  }
}
```

```
> deuxieme_degre(1,-5,6)
x1 = 2
x2 = 3
```

```
> deuxieme_degre(1,0,1)
Pas de solution
```

6.3 Les objets de Rcpp sont passés par référence

La fonction suivante le démontre : les vecteurs de Rcpp sont passés par référence (ou sont des pointeurs, comme vous préférez). Ceci permet un grand gain de temps, en évitant la copie des données, mais a aussi des effets potentiellement indésirables.

```
#include <Rcpp.h>
// Création d'un vecteur (initialisé à 0)
// [[Rcpp::export]]
Rcpp::NumericVector doubler(Rcpp::NumericVector x) {
  int n = x.size();
  for(int i = 0; i < n; i++) {
    x[i] *= 2;
  }
  return x;
}
```

```
> x <- seq(0,1,0.1);
> x
 [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
> doubler(x)
 [1] 0.0 0.2 0.4 0.6 0.8 1.0 1.2 1.4 1.6 1.8 2.0
```

```
> x # oups
 [1] 0.0 0.2 0.4 0.6 0.8 1.0 1.2 1.4 1.6 1.8 2.0
```

De plus, quand on copie un objet en R, comme ci-dessous avec `y <- x`, la recopie des données n'a pas lieu dans un premier temps. Au début, `y` pointe vers le même objet que `x` ; la recopie n'aura lieu que si on modifier `y`. Quand on utilise `Rcpp` ce mécanisme n'est plus actif:

```
> x <- seq(0,1,0.1);
> y <- x
> doubler(y)
 [1] 0.0 0.2 0.4 0.6 0.8 1.0 1.2 1.4 1.6 1.8 2.0
```

```
> x
 [1] 0.0 0.2 0.4 0.6 0.8 1.0 1.2 1.4 1.6 1.8 2.0
```

```
> y
 [1] 0.0 0.2 0.4 0.6 0.8 1.0 1.2 1.4 1.6 1.8 2.0
```

La prudence est de mise! Notez qu'on a aussi le comportement suivant:

```
> x <- 1:5;
> x
[1] 1 2 3 4 5
```

```
> doubler(x)
[1]  2  4  6  8 10
```

```
> x
[1] 1 2 3 4 5
```

Utiliser `typeof(x)` pour résoudre cette énigme.

À supposer que ce comportement soit indésirable, comment y remédier ?

```
#include <Rcpp.h>
// Création d'un vecteur (initialisé à 0)
// [[Rcpp::export]]
Rcpp::NumericVector doubler2(Rcpp::NumericVector x) {
  int n = x.size();
  // contrairement à ce qu'on pense, ceci ne copie pas x
  Rcpp::NumericVector y = x;
  for(int i = 0; i < n; i++) y[i] *= 2;
  return y;
}

// [[Rcpp::export]]
Rcpp::NumericVector doubler3(Rcpp::NumericVector x) {
  int n = x.size();
  // il faut utiliser clone
  Rcpp::NumericVector y = Rcpp::clone(x);
  for(int i = 0; i < n; i++) y[i] *= 2;
  return y;
}
```

```
> x <- seq(0,1,0.1); invisible(doubler2(x)); x
 [1] 0.0 0.2 0.4 0.6 0.8 1.0 1.2 1.4 1.6 1.8 2.0
```

```
> x <- seq(0,1,0.1); invisible(doubler3(x)); x
 [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

6.4 Surcharge

En C++ deux fonctions peuvent avoir le même nom, à condition que le type des arguments soit différent, ce qui permet au compilateur de les différentier. À noter : le type retourné par la fonction n'est pas pris en compte dans ce mécanisme.

Ceci s'appelle la *surcharge* des fonctions. On a également une surcharge des opérateurs (comme +, <<, etc).

Attention ! On ne peut pas exporter des fonctions surchargées avec Rcpp. Il faut donc maintenir ce mécanisme dans la partie strictement C++ de votre code.

Voici un exemple de fonction surchargée.

```
#include <Rcpp.h>
using namespace Rcpp;

double d2(double a, double b) {
  return (a*a + b*b);
}

int d2(int a, int b) {
  return (a*a + b*b);
}

//[[Rcpp::export]]
void exemple_d2() {
  double x = 1.0;
  double y = 2.4;
  double z = d2(x, y);
  Rcout << "d2(x,y) = " << z << std::endl;
  int u = 1;
  int v = 4;
  int w = d2(u, v);
  Rcout << "d2(u,v) = " << w << std::endl;
}
```

```
> exemple_d2()
d2(x,y) = 6.76
d2(u,v) = 17
```

On peut également avoir des fonctions qui ont le même nom, et un nombre différent d'arguments.

6.5 Templates

Les templates facilitent la réutilisation de code avec des types différents. Prendre le temps d'utiliser des méthodes propres pour éviter les copier-coller dans le code est de toute première importance quand il s'agit de « maintenir » le code.

6.5.1 Fonctions polymorphes

L'exemple ci-dessous définit un template `d2` ; lors de la compilation de la fonctions `addition_int` et `addition_double` ce template sera instancié avec `TYPE = int` puis avec `TYPE = double`.

```
#include <Rcpp.h>
using namespace Rcpp;

template<typename TYPE>
TYPE d2(TYPE a, TYPE b) {
  return a*a + b*b;
}

//[[Rcpp::export]]
```

```
void exemple_template_d2() {
  double x = 1.0;
  double y = 2.4;
  double z = d2(x, y);
  Rcout << "d2(x,y) = " << z << std::endl;
  int u = 1;
  int v = 4;
  int w = d2(u, v);
  Rcout << "d2(u,v) = " << w << std::endl;
}
```

Exemple:

```
> exemple_template_d2()
d2(x,y) = 6.76
d2(u,v) = 17
```

Si le compilateur peinait à trouver par quoi il faut remplacer `TYPE`, il serait possible de préciser en tapant par exemple `d2<float>`.

On peut donner un autre exemple en faisant un template pour la fonction `swap` que nous avions écrite il y a quelques chapitres.

```
template<typename TYPE>
void swap(TYPE & a, TYPE & b) {
  TYPE tmp = a;
  a = b;
  b = tmp;
}
```

Note : n'utilisez pas ce template dans votre code, C++ a déjà un template `std::swap`, qui est mieux fait que celui-ci ! (Il évite la copie). Voir https://en.cppreference.com/w/cpp/algorithm/swap

6.5.2 Laisser le compilateur s'occuper des types

Une des utilités des templates est de permettre de ne pas trop se soucier de types « compliqués » comme ceux des fonctions. Ci-dessous on définit une fonction `num_derive` qui donne une approximation de la dérivée de son premier argument.

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
#define epsilon 0.001

template<typename TYPE, typename FTYPE>
TYPE num_derive(FTYPE f, TYPE a) {
  return (f(a+epsilon) - f(a-epsilon))/(2.0*epsilon);
}

double square(double a) {
  return a*a;
}

//[[Rcpp::export]]
void test1(double x) {
  double fx = square(x);
  double dfx = num_derive(square, x);
```

```
  SHOW(fx);
  SHOW(dfx);
}
```

```
> test1(3)
fx = 9
dfx = 6
```

On voit que `num_derive` prend deux arguments, une fonction `f` et un point `a`, et donne une approximation de $f'(a)$ en calculant

$$\frac{f(a+\varepsilon)-f(a-\varepsilon)}{2\varepsilon}.$$

Quel est le type d'une fonction ?! Nous laissons au compilateur le soin de remplacer `FTYPE` par la bonne valeur. Dans notre exemple, lors d'instanciation du template, `TYPE` est remplacé par `double` et `FTYPE` par `double (*)(double)` (pointeur vers une fonction qui renvoie un `double` et dont l'argument est un `double`). Un code équivalent serait

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
#define epsilon 0.001
double num_derive(double (*f)(double), double a) {
  return (f(a+epsilon) - f(a-epsilon))/(2*epsilon);
}

double square(double a) {
  return a*a;
}

//[[Rcpp::export]]
void test_derive(double x) {
  double fx = square(x);
  double dfx = num_derive(square, x);
  SHOW(fx);
  SHOW(dfx);
}
```

```
> test_derive(3)
fx = 9
dfx = 6
```

6.6 C++11

Le standard C++11 est maintenant accepté dans les packages R. Pour permettre la compilation avec les extensions offertes par C++11, vous devez faire :

```
> Sys.setenv("PKG_CXXFLAGS" = "-std=c++11")
```

6.6.1 Les boucles sur un vecteur et `auto`

Parmi les extensions offertes, les plus séduisantes sont sans doute : le mot-clef `auto`, qui laisse le compilateur deviner le type, quand le contexte le permet, de façon similaire à ce qui est fait dans les templates, et les boucles `for` qui parcourent un vecteur… comme en R.

```
#include <Rcpp.h>
//[[Rcpp::export]]
int count_zeros3(Rcpp::IntegerVector x) {
  int re = 0;
  for(auto a : x) {
    re += (a == 0)?1:0;
  }
  return re;
}
```

```
> set.seed(1); a <- sample(0:99, 1e6, TRUE)
> count_zeros3(a);
[1] 9816
```

Si on veut pouvoir modifier la valeur des éléments du vecteur, il suffit d'utiliser une référence :

```
#include <Rcpp.h>
//[[Rcpp::export]]
void add_one(Rcpp::IntegerVector x) {
  for(auto & a : x) {
    a++;
  }
}
```

```
> a <- 1:10
> add_one(a)
> a
 [1]  2  3  4  5  6  7  8  9 10 11
```

NOTE En utilisant une référence, on évite la copie. Même si on ne désire pas modifier les éléments du vecteur, cela peut modifier énormément les performances de la boucle en question !

TODO ajouter exemple !!!

6.6.2 Nombres spéciaux

C++11 propose aussi des fonctions pour tester si des nombres flottants sont finis ou non, sont `NaN`. Il est aussi plus facile de renvoyer un `NaN` ou une valeur infinie (c'était possible avec `std::numeric_limits`).

```
#include <Rcpp.h>
#include <cmath>       // !!! CE HEADER EST NÉCESSAIRE
//[[Rcpp::export]]
void tests(double x) {
  if(std::isnan(x)) Rcpp::Rcout << "NaN\n";
  if(std::isinf(x)) Rcpp::Rcout << "infini\n";
  if(std::isfinite(x)) Rcpp::Rcout << "fini\n";
}

//[[Rcpp::export]]
Rcpp::NumericVector specials() {
  Rcpp::NumericVector x(2);
  x[0] = NAN;
  x[1] = INFINITY;
  return x;
}
```

```
> tests( NA )
NaN
```

```
> tests( -Inf )
infini
```

```
> tests( pi )
fini
```

```
> specials();
[1] NaN Inf
```

Chapitre 7

Exemple : Metropolis-Hastings (2)

Appliquons ce que nous avons appris sur les templates au chapitre précédent pour rendre possible la réutilisation de notre code pour l'algorithme de Metropolis-Hastings avec d'autres fonctions.

7.1 Une version templatée

On repart de la fonction présentée plus haut. On construit un fichier `Metropolis.h` qui contient

```
#include <Rcpp.h>
using namespace Rcpp;

#ifndef METROPOLISHASTINGS
#define METROPOLISHASTINGS

template<typename FTYPE>
NumericMatrix Metropolis(FTYPE Pi, int B, double sd, int burn = 0, int thin = 1) {
  NumericMatrix R(B, 2);
  thin = (thin < 1)?1:thin;
  double x1 = 0.0, x2 = 0.0;
  int b = 0;
  double pi_x = Pi(x1, x2);
  for(int k = 0; b < B ; k++) { // !! boucle exotique !!
    double y1 = x1 + R::rnorm(0,sd);
    double y2 = x2 + R::rnorm(0,sd);
    double pi_y = Pi(y1, y2) / Pi(x1, x2);
    double u = R::unif_rand();
    if(u * pi_x < pi_y) {
      x1 = y1;
      x2 = y2;
      pi_x = pi_y;
    }
    if(k > burn && (k % thin) == 0) {
      R(b,0) = x1;
      R(b,1) = x2;
      b++;
    }
    if((k % 1000) == 0) // toutes les mille itérations
```

```
      checkUserInterrupt();
  }
  return R;
}
#endif
```

Tout ce que nous avons fait, c'est d'inclure la fonction `Pi` parmi la liste des paramètres de la fonction. Nous laissons au compilateur le soin de remplacer `FTYPE` par la bonne valeur.

Ce template reste très perfectible. Il serait bon par exemple de pouvoir également modifier le point de départ (toujours, ici, `x1 = 0, x2 = 0`), cela ne serait pas très compliqué.

Les directives de compilation `#ifndef` permettent d'éviter, si le fichier est inclus deux fois, de redéfinir le template. Prenez l'habitude de protéger d'emblée le contenu de tous vos fichiers `.h` par ce mécanisme: cela vous épargnera d'avoir à le faire quand la complexité de votre projet augmentera.

Notons que pour vraiment obtenir une fonction généraliste, il faudrait (entre autres choses que l'utilisateur puisse spécifier un point de départ à l'algorithme (ici c'est toujours $(0,0)$).

7.2 Mise en œuvre

```
#include <Rcpp.h>
#include "metropolis.h"
using namespace Rcpp;

double Pi(double x1, double x2) {
  return pow((1 + x1*x1 + x1*x2 + x2*x2),-3.0);
}

//[[Rcpp::export]]
NumericMatrix MH_cpp_2(int B, double sd, int burn = 0, int thin = 1) {
  return Metropolis(Pi, B, sd, burn, thin);
}
```

Chapitre 8

Chaînes de caractères

Notions de code Ascii. Iso-8859-1 et variantes. UTF-8.

Les codes 0 à 31 correspondent à des codes de contrôles (par exemple 8 pour 'backspace') et ne sont pas imprimables ; c'est aussi le cas du caractère 127 qui correspond à 'del'.

Affichons le code Ascii de 32 à 127 sur 8 colonnes.

```
#include <Rcpp.h>
#define COLS 8
#define OFF (96/COLS)
//[[Rcpp::export]]
void ascii() {
  for(int i = 32;;i++) {
    for(int k = 0; k < COLS; k++) {
       Rprintf("%03d : ",i + k*OFF);
       if(i + k*OFF < 127)
         Rcpp::Rcout << (char) (i + k*OFF) << "    ";
       else {
         Rcpp::Rcout << "DEL" << std::endl;
         return;
       }
    }
    Rcpp::Rcout << std::endl;
  }
}
```

```
> ascii()
## 032 :      044 : ,    056 : 8    068 : D    080 : P    092 : \    104 : h    116 : t
## 033 : !    045 : -    057 : 9    069 : E    081 : Q    093 : ]    105 : i    117 : u
## 034 : "    046 : .    058 : :    070 : F    082 : R    094 : ^    106 : j    118 : v
## 035 : #    047 : /    059 : ;    071 : G    083 : S    095 : _    107 : k    119 : w
## 036 : $    048 : 0    060 : <    072 : H    084 : T    096 : `    108 : l    120 : x
## 037 : %    049 : 1    061 : =    073 : I    085 : U    097 : a    109 : m    121 : y
## 038 : &    050 : 2    062 : >    074 : J    086 : V    098 : b    110 : n    122 : z
## 039 : '    051 : 3    063 : ?    075 : K    087 : W    099 : c    111 : o    123 : {
## 040 : (    052 : 4    064 : @    076 : L    088 : X    100 : d    112 : p    124 : |
## 041 : )    053 : 5    065 : A    077 : M    089 : Y    101 : e    113 : q    125 : }
## 042 : *    054 : 6    066 : B    078 : N    090 : Z    102 : f    114 : r    126 : ~
## 043 : +    055 : 7    067 : C    079 : O    091 : [    103 : g    115 : s    127 : DEL
```

8.1 Chaînes de la vieille école : `C strings`

Ce sont des pointeurs vers un entier de type `char`, c'est-à-dire codé sur 8 bits – à moins de travailler sur une architecture très exotique – et pouvant prendre des valeurs de 0 à 255. La chaîne de caractère est un tableau dont la fin est signalée par la présence d'une valeur nulle.

EN C et C++, la syntaxe `'a'` (guillement simple) implique le remplacement de la lettre par son code ascii (pour `a` c'est 97). Dans l'exemple ci-dessous, la chaîne 'b' est explicitement présentée comme le tableau contenant 116, 111, 105, 0. Les lettres de code ascii qui correspondent sont `t`, `o` et `i` ; si on avait écrit `char b[] = "toi";` le résultat aurait été identique.

```
#include <Rcpp.h>
//[[Rcpp::export]]
void quelques_chaines() {
  char a[] = "Bonjour";
  char b[4] = {116, 111, 105, 0};      // 3 lettres et un 0 final
  char c[5] = {'v', 'o', 'u', 's', 0}; // 4 lettres et un 0 final
  Rcpp::Rcout << a << " " << b << std::endl;
  Rcpp::Rcout << a << " " << c << std::endl;
  Rcpp::Rcout << "Taille de a = " << sizeof(a) << std::endl;
  Rcpp::Rcout << "Longueur de la chaîne = " << std::strlen(a) << std::endl;
  Rcpp::Rcout << "a[0] = " << a[0] << std::endl;
  Rcpp::Rcout << "a[1] = " << a[1] << std::endl;
}
```

```
> quelques_chaines()
Bonjour toi
Bonjour vous
Taille de a = 8
Longueur de la chaîne = 7
a[0] = B
a[1] = o
```

Notez `std::strlen(a)` qui renvoie la longueur de la chaîne `a`.

Comme ce sont des pointeurs, on ne va pas bêtement tester leur égalité... Il faut utiliser `std::strcmp(a,b)` qui renvoie 0 si les chaînes sont égales, et dans le cas contraire un nombre dont le signe est positif si $a < b$ dans l'ordre lexicographique, négatif sinon.

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
//[[Rcpp::export]]
void compare() {
  char a[] = "Gaston";
  char b[] = "Fantasio";
  char c[] = "Gaston";
  SHOW(a == b);
  SHOW(a == c);
  SHOW( std::strcmp(a,b) );
  SHOW( std::strcmp(b,a) );
  SHOW( std::strcmp(a,c) );
}
```

```
> compare()
a == b = 0
a == c = 0
std::strcmp(a,b) = 1
```

```
std::strcmp(b,a) = -1
std::strcmp(a,c) = 0
```

8.1.1 Manipuler les chaînes de R comme des *C strings*

La fonction `CHAR`, appliquée à un élément d'un `CharacterVector`, renvoie un `const char *`. Le mot clef `const` devant un type indique au compilateur que le contenu ne peut être modifié : un `const char` est `char` « constant » ; le pointeur renvoyé par `CHAR` pointe vers des `const char`, c'est-à-dire vers une chaîne de caractères que l'utilisateur n'a pas le droit de modifier. On peut tricher en utilisant un cast vers un `char *` mais c'est à nos risques et périls…

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
void c_strings(CharacterVector x) {
  int n = x.size();
  for(int i = 0; i < n; i++) {
    const char * a = CHAR(x[i]);
    Rcout << i << " : " << a << std::endl;
    Rcout << " adresse : " << (void *) a << std::endl;
  }
}
```

```
> a <- c("gaston", "jeanne", "gaston")
> c_strings(a)
0 : gaston
 adresse : 0x55f0c9e76ce0
1 : jeanne
 adresse : 0x55f0c9e76d50
2 : gaston
 adresse : 0x55f0c9e76ce0
```

On s'aperçoit ici que les deux `"gaston"` sont stockés par R à la même adresse (les apprentis sorciers qui auraient joué avec `.Internal(inspect(a))` l'auront constaté). Ceci incite à être très prudent, toute modification *in place* du premier modifierait l'autre ! Ça n'est pas pour rien que `CHAR` renvoie un `const char *` et non un `char *`.

8.1.2 Exemple d'utilisation des C strings

C'est rustique (et risqué) mais ça peut être redoutablement efficace. Si on sait que les codes ascii des nombres de 0 à 9 se suivent…

```
#include <Rcpp.h>
inline int fast_atoi(const char * a)
{
    int re = 0;
    while( *a ) {
        re = re*10 + (*(a++) - '0');
    }
    return re;
}
//[[Rcpp::export]]
Rcpp::IntegerVector as_int(Rcpp::CharacterVector x) {
  int n = x.size();
```

```
  Rcpp::IntegerVector R(n);
  for(int i = 0; i < n; i++) {
    const char * a = CHAR(x[i]);
    R[i] = fast_atoi(a);
  }
  return R;
}
```

Cette fonction file comme le vent mais ne fait rien pour vérifier que la chaîne code correctement un entier !

```
> as_int( c("456", "123", "789", "Dave Brubeck") )
[1]        456        123        789 1069870157
```

Autres possibilités offertes pas les C-strings : modifier une chaîne *in place* en en remplaçant les espaces par des 0 (code de fin de chaîne) pour en isoler les mots, etc. Cela peut aller beaucoup plus vite que les opérations équivalentes avec les chaînes C++.

8.2 Chaînes C++

La librairie standard contient la classe `string` qui facilite grandement le travail.

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
//[[Rcpp::export]]
void autres_chaines() {
  std::string x = "ça va ?";
  std::string y = "Bonjour";
  std::string z = y + " " + x; // Concaténation
  SHOW(x);
  SHOW(y);
  SHOW(y[0]);
  SHOW(y[1]);
  SHOW(z);
}
```

```
> autres_chaines()
x = ça va ?
y = Bonjour
y[0] = B
y[1] = o
z = Bonjour ça va ?
```

La comparaison est grandement facilitée :

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
//[[Rcpp::export]]
void compare2() {
  std::string a = "Gaston";
  std::string b = "Fantasio";
  std::string c = "Gaston";
  SHOW(a == b);
  SHOW(a == c);
  SHOW(a < b);
```

```
  SHOW(b < c);
}
```

```
> compare2()
a == b = 0
a == c = 1
a < b = 0
b < c = 1
```

8.2.1 Passer d'un style à l'autre

Cette fonction démontre comment passer facilement d'un style de chaîne à l'autre :

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
//[[Rcpp::export]]
void conversions() {
  char a[] = "Gaston";
  std::string b(a);
  SHOW(a);
  SHOW(b);

  std::string x = "Longtarin";
  const char * y = x.c_str();
  SHOW(x);
  SHOW(y);
}
```

```
> conversions()
a = Gaston
b = Gaston
x = Longtarin
y = Longtarin
```

8.2.2 Manipuler les chaînes de R comme des `std::string`

On peut pour commencer les passer simplement en argument d'une fonction :

```
#include <Rcpp.h>
//[[Rcpp::export]]
void perroquet(std::string x) {
  Rcpp::Rcout << "Vous dites : " << x << std::endl;
  Rcpp::Rcout << "longueur = " << x.size() << std::endl;
  Rcpp::Rcout << "adresse &x[0] = " << (void *) &x[0] << std::endl;
  Rcpp::Rcout << "adresse x.c_str() = " << (void *) x.c_str() << std::endl;
}
```

Le `(void *)` est nécessaire pour faire afficher l'adresse du pointeur : si on fournit un objet de type `char *` à l'opérateur `<<` il considère qu'on veut afficher la chaîne de caractère. Essayez `Rcpp::Rcout << y` si nécessaire.

```
> perroquet("bavard")
Vous dites : bavard
longueur = 6
```

```
adresse &x[0] = 0x7ffcd4e5cc58
adresse x.c_str() = 0x7ffcd4e5cc58
```

La fonction `as<std::string>` permet la conversion vers une `std::string` (notez la présence des chevrons <> : `as` est une fonction templatée).

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
void cpp_strings(CharacterVector x) {
  int n = x.size();
  for(int i = 0; i < n; i++) {
    std::string a = as<std::string>(x[i]);
    Rcout << i << " : " << a << std::endl;
  }
}
```

Application :

```
> a <- c("gaston", "jeanne", "gaston")
> cpp_strings(a)
0 : gaston
1 : jeanne
2 : gaston
```

8.3 Chaines `Rcpp::String`

Les `CharacterVector` sont des vecteurs de chaînes de caractères. On a rarement besoin de manipuler leurs éléments séparément autrement qu'en les convertissant en `char *` ou en `std::string`, comme on l'a fait plus haut ; cependant la classe `Rcpp::String` permet de le faire. Voici un bref exemple.

```
#include <Rcpp.h>
#define SHOW(x) Rcpp::Rcout << #x << " = " << (x) << std::endl;
using namespace Rcpp;
//[[Rcpp::export]]
void atomic_strings(CharacterVector x) {
  int n = x.size();
  for(int i = 0; i < n; i++) {
    String a = x[i];
    const char * b = a.get_cstring();      // équivaut à b = CHAR(x(i));
    Rcout << i << " : " << b << std::endl;
  }
}
```

```
> atomic_strings(a)
0 : gaston
1 : jeanne
2 : gaston
```

8.4 Exemple : lecture d'un fichier

Quand on aura vu les vecteurs standards !

(TODO)

8.5 Addendum sur les listes

Dans le cas des vecteurs nommés ou des listes (ou des data frame), on peut utiliser indifféremment

Les fonctions ci-dessous ne gèrent pas les erreurs, voir dans le chapitre sur les listes la fonction `get_alpha`.

```
#include <Rcpp.h>
using namespace Rcpp;

//[[Rcpp::export]]
NumericVector get_from_list_1(List L, std::string k) {
  NumericVector R = L[k];
  return R;
}
```

Pour en finir avec le sujet un dernier exemple de traitement du contenu d'une liste : énumérer les éléments, leur nom, et leur type.

```
#include <Rcpp.h>
using namespace Rcpp;

std::string le_type(SEXP x) {
  switch( TYPEOF(x) ) {
    case INTSXP:
      return "integer";
    case REALSXP:
      return "double";
    case LGLSXP:
      return "logical";
    case STRSXP:
      return "character";
    case VECSXP:
      return "list";
    case NILSXP:
      return "NULL";
    default:
      return "autre";
  }
}

//[[Rcpp::export]]
void contenu_liste(List L) {
  int n = L.size();
  if(n == 0) {
    Rcout << "liste vide\n";
    return;
  }
  SEXP R = L.names();
  bool has_names = (TYPEOF(R) == STRSXP);
  for(int i = 0; i < n; i++) {
    Rcout << i ;
    if(has_names)
```

```
      if( strcmp( CHAR(STRING_ELT(R, i)), "" ) )
        Rcout << " \"" << CHAR(STRING_ELT(R, i)) << "\"";
    Rcout << " : " << le_type(L[i]) << std::endl;
  }
}
```

Notez l'usage de `STRING_ELT(R, i)` : si `R` était un `CharacterVector` on pourrait écrire `R[i]`, mais on a ici utilisé un `SEXP`, ce qui oblige à utiliser cette fonction un peu inélégante.

Chapitre 9

Exemple : chaînes de caractère et spécialisation de template

Saisissons l'occasion de la manipulation de chaînes de caractères pour illustrer la possibilité de donner des implémentations différentes d'un même template.

La conversion d'une `std::string` vers un `double` se fait avec `std::stod`, vers un `int` avec `std::stoi`, vers un `float` avec `std::stof`. Le template ci-dessous permet d'utiliser `sto<double>`, `sto<int>`, `sto<float>`.

Note l'argument `const std::string & x` est une référence à une chaîne `std::string` *constante* ; ceci indique au compilateur que la fonction ne modifiera jamais `x`. Cela peut permettre des optimisations à la compilation.

```
#ifndef STO_
#define STO_
// Déclaration du template
template<typename T>
T sto(const std::string & x);

// Implémentations pour les types 'double', '
template<>
double sto<double>(const std::string & x) {
  return std::stod(x);
}

template<>
float sto<float>(const std::string & x) {
  return std::stof(x);
}

template<>
int sto<int>(const std::string & x) {
  return std::stoi(x);
}
#endif
```

L'intérêt d'une telle définition n'est pas forcément apparent. Un des avantages est qu'un tel template peut être appelé dans un autre template plus complexe. Par exemple :

```
template<typename T>
T somme(std::string a, std::string b) {
```

```
  return sto<T>(a) + sto<T>(b);
}
```

Chapitre 10

Matrices

Les matrices de Rcpp n'ont pas les opérations de l'algèbre linéaire (produit matriciel, inversion de matrice). Il faut faire appel à d'autres packages, comme RcppArmadillo ou RcppEigen pour accéder à de telles fonctionnalités.

Commençons par une généralité sur la façon dont les matrices de Rcpp sont stockées en mémoire.

10.1 Le *column major mode*

```
#include <Rcpp.h>
using namespace Rcpp;
//[[Rcpp::export]]
void affiche_matrice(NumericMatrix x) {
  int n = x.nrow(); //dimensions
  int m = x.ncol();
  for(int i = 0; i < n; i++) {
    for(int j = 0; j < m; j++) {
      Rcout << "(" << i << "," << j << ")" << " = " << x(i,j) << "\n";
    }
  }
}
//[[Rcpp::export]]
void affiche_matrice2(NumericMatrix x) {
  int s = x.size();
  for(int i = 0; i < s; i++)
    Rcout << x[i] << " ";      // ou bien for(auto a: x) Rcout << a << " ";
  Rcout << "\n";
}
```

Il y a une conséquence : lire tous les éléments d'une matrice colonne par colonne va beaucoup plus vite que les lire ligne par ligne!

TODO exemple de la somme de tous les éléments d'une (grande) matrice

10.2 Matrices avec RcppArmadillo

Il faut installer le package RcppArmadillo pour pouvoir utiliser cette librairie avec Rcpp. La documentation de la librairie Armadillo est à l'adresse http://arma.sourceforge.net/docs.html

10.2.1 Conversions des vecteurs et matrices de R

Les vecteurs et matrices de RcppArmadillo sont des arma::vec et des arma::mat. Les opérations d'addition et de produit se font simplement avec les opérateurs + et *. Rcpp se débrouille pour faire une conversion « à la volée ». La fonction suivante fait le produit de la matrice A par le vecteur v:

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
#include <Rcpp.h>
//[[Rcpp::export]]
arma::vec arma_produit_mv(arma::mat A, arma::vec v) {
  return A*v;
}
```

```
> A <- matrix( c(1,2,0,1), nrow = 2)
> v <- c(1, 4)
> arma_produit_mv(A, v)
     [,1]
[1,]    1
[2,]    6
```

```
> A %*% v
     [,1]
[1,]    1
[2,]    6
```

10.2.2 Éviter la copie

Le problème de cette façon de procéder c'est que la conversion des objets R vers les objets Armadillo implique une copie du contenu des matrices et des vecteurs, ce qui ralentit énormément l'exécution.

La solution est de convertir non pas vers des arma::mat et des arma::vec, mais vers des références à de tels objets!

Ci-dessous les fonctions foo1 et foo2 somment le premier élément des deux matrices passées en argument, avec ou sans passage par référence ; les deux fonctions qui suivent vont permettre de comparer les performances des deux méthodes ; la dernière démontre la possibilité d'attraper les matrices passées par R directement comme des références.

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
#include <Rcpp.h>

using namespace Rcpp;

double foo1(arma::mat a, arma::mat b){
  return a(0,0) + b(0,0);
}
```

```
double foo2(const arma::mat & a, const arma::mat & b){
  return  a(0,0) + b(0,0);
}

// [[Rcpp::export]]
double test_1(arma::mat A, arma::mat B) {
  return foo1(A, B);
}

// [[Rcpp::export]]
double test_2(arma::mat A, arma::mat B) {
  return foo2(A, B);
}

// [[Rcpp::export]]
double test_3(arma::mat & A, arma::mat & B) {
  return foo2(A, B);
}
```

```
> x <- matrix(rnorm(1e4),100,100)
> y <- matrix(rnorm(1e4),100,100)
> mbm <- microbenchmark::microbenchmark(test_1(x, y), test_2(x, y), test_3(x,y),
+                                       times = 1e3)
> mbm
Unit: microseconds
         expr    min      lq      mean median      uq     max neval
 test_1(x, y) 15.212 15.8735 17.131617 16.179 16.5415 842.381  1000
 test_2(x, y)  8.943  9.3435 10.203887  9.512  9.7040 592.884  1000
 test_3(x, y)  1.312  1.7085  2.514989  1.836  1.9885 590.784  1000
```

On voit l'intérêt de ne pas faire de copie !

Attention, malgré cela, il est possible que si vous modifiez la valeur d'un argument `x`, par exemple par un calcul du type `x = A*x;`, Armadillo change la place de `x` en mémoire... Bref il n'est pas évident de contrôler quels changements vont être apportés « in place » à une matrice de R passée à `RcppArmadillo`.

Une dernière chose : si dans une fonction, vous voulez définir une matrice `arma::mat` à partir d'une `NumericMatrix` *sans faire de copie*, voici comment procéder :

```
//[[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
// [[Rcpp::export]]
void no_copy(Rcpp::NumericMatrix A) {
  arma::mat a(A.begin(), A.nrow(), A.ncol(), false);
  Rcpp::Rcout << A(0,0) << "\n";
  Rcpp::Rcout << a(0,0) << "\n"; // les mêmes éléments...
}
```

10.2.3 Inversion de matrice quelconque

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;
// [[Rcpp::export]]
```

```
arma::mat arma_inverse(arma::mat & A) {
  return A.i();
}
```

```
> set.seed(2); A <- matrix( runif(100), 10, 10)
> range(arma_inverse(A) - solve(A))
[1] -1.131040e-15  1.221245e-15
```

10.2.4 Inversion de matrice symétrique définie positive

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;
// [[Rcpp::export]]
arma::mat arma_pds_inverse(arma::mat & A) {
  return inv_sympd(A);
}
```

```
> set.seed(7); B <- crossprod(matrix( runif(1e6), 100, 100))
> range(solve(B) - arma_pds_inverse(B))
[1] -1.030116e-07  9.039923e-08
```

10.3 Matrices avec `RcppEigen`

Il faut installer le package `RcppEigen` pour pouvoir utiliser cette librairie avec Rcpp. La documentation de la librairie Eigen est à l'adresse http://eigen.tuxfamily.org/index.php et plus précisément http://eigen.tuxfamily.org/dox/

10.3.1 Eviter la copie

Les Map Matrix

10.3.2 Quelques opérations

```
// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>
using namespace Rcpp;
using namespace Eigen;
typedef Eigen::Map<Eigen::VectorXd> VEC;
typedef Eigen::Map<Eigen::MatrixXd> MAT;

// [[Rcpp::export]]
void g(NumericMatrix a_) {
  MAT a(as<MAT>(a_));
  Rcout << "a(0,0) = " << a(0,0) << "\n";
  a(0,0)++;
  Rcout << "a(0,0) = " << a(0,0) << "\n";
}
```

```
// [[Rcpp::export]]
NumericMatrix eigen_produit(NumericMatrix a_, NumericMatrix b_) {
  MAT a(as<MAT>(a_));
  MAT b(as<MAT>(b_));
  MatrixXd c = a*b;
  return wrap(c);
}

// [[Rcpp::export]]
NumericVector eigen_produit2(NumericMatrix a_, NumericVector b_) {
  MAT a(as<MAT>(a_));
  VEC b(as<VEC>(b_));
  VectorXd c = a*b;
  return wrap(c);
}
```

10.3.3 L'utilisation de `.noalias()`

TBD

10.3.4 Inversion de matrice quelconque

```
// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>
using namespace Rcpp;
using namespace Eigen;
typedef Eigen::Map<Eigen::VectorXd> VEC;
typedef Eigen::Map<Eigen::MatrixXd> MAT;

// [[Rcpp::export]]
NumericMatrix eigen_inverse(NumericMatrix a_) {
  MAT a(as<MAT>(a_));
  int n = a.rows();
  if(a.cols() != n)
    stop("Dimensions mismatch\n");
  MatrixXd c(a.inverse());
  return wrap(c);
}
```

```
> set.seed(2); A <- matrix( runif(100), 10, 10)
> range(eigen_inverse(A) - solve(A))
[1] -4.440892e-15  3.552714e-15
```

10.3.5 Inversion de matrice symétrique définie positive

Il y a plusieurs façon d'inverser une matrice symétrique définie positive. Passer par la décomposition de Choleski $A = LL^t$ est réputée être la méthode la plus rapide.

```
// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>
```

```
using namespace Rcpp;
using namespace Eigen;
typedef Eigen::Map<Eigen::VectorXd> VEC;
typedef Eigen::Map<Eigen::MatrixXd> MAT;

// [[Rcpp::export]]
NumericMatrix eigen_pds_inverse(NumericMatrix a_) {
  MAT a(as<MAT>(a_));
  int n = a.rows();
  if(a.cols() != n)
    stop("Dimensions mismatch\n");
  MatrixXd c(a.llt().solve(MatrixXd::Identity(n,n)));
  return wrap(c);
}
```

```
> set.seed(7); B <- crossprod(matrix( runif(1e6), 100, 100))
> range(solve(B) - eigen_pds_inverse(B))
[1] -1.852513e-07  1.636611e-07
```

10.4 Benchmarking (1)

Faisons le benchmarking de ces opérations élémentaires.

```
> set.seed(2); A <- matrix( runif(100), 10, 10)
> mbm <- microbenchmark::microbenchmark( arma_inverse(A), eigen_inverse(A), solve(A))
> mbm
Unit: microseconds
             expr    min      lq     mean  median      uq     max neval
  arma_inverse(A)  4.000  4.2780 12.56304  4.6655  4.8900 795.690   100
 eigen_inverse(A)  3.709  4.0645  9.99101  4.5515  4.8105 558.086   100
         solve(A) 14.507 15.6495 16.59988 16.4500 16.8570  49.876   100
```

```
> set.seed(7); B <- crossprod(matrix( runif(1e6), 1000, 1000))
> mbm <- microbenchmark::microbenchmark( eigen_pds_inverse(B), arma_pds_inverse(B), times = 5)
> mbm
Unit: milliseconds
                 expr      min       lq     mean   median       uq      max neval
 eigen_pds_inverse(B) 186.7043 187.4374 188.2719 188.4244 189.1168 189.6768     5
  arma_pds_inverse(B) 311.8720 312.3313 312.7228 312.7966 312.8597 313.7546     5
```

10.5 Benchmarking sur l'algorithme Nipals

Bla bla Nipals

10.5.1 Implémentation avec Armadillo:

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace Rcpp;
```

```
//[[Rcpp::export]]
arma::vec arma_nipals(arma::mat & a, arma::vec & x, int n) {
  for(int i = 0; i < n; i++)  {
    x = a*x;
    x /= norm(x);
  }
  return x;
}
```

10.5.2 Implémentation avec Eigen:

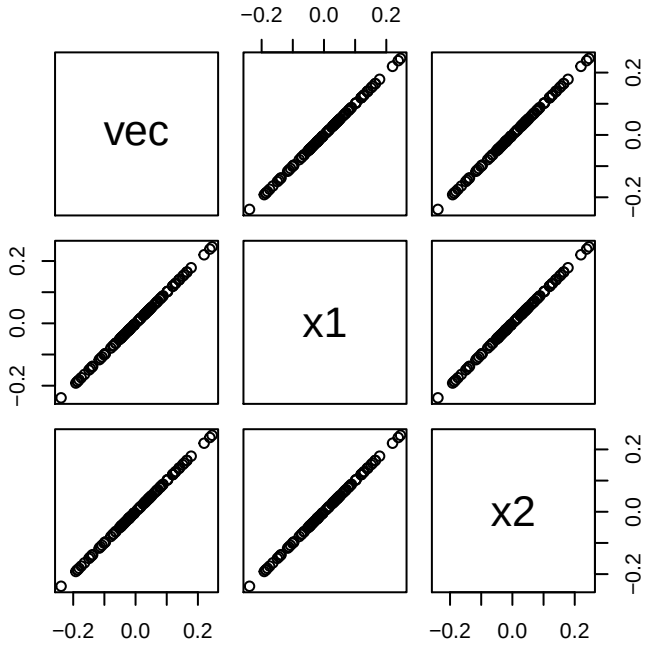
Avec une Map Matrix, Eigen modifie le `NumericVector x` en place (contrairement à ce que fait l'implémentation avec Armadillo ci-dessus, malgré le passage par référence). Il est donc inutile de renvoyer le résultat.

```
// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>

using namespace Rcpp;
using namespace Eigen;
typedef Eigen::Map<Eigen::VectorXd> VEC;
typedef Eigen::Map<Eigen::MatrixXd> MAT;

// [[Rcpp::export]]
void eigen_nipals(MAT a, VEC x, int n) {
  for(int i = 0; i < n; i++) {
    x = a*x;
    x /= x.norm();
  }
}
```

10.5.3 Comparaison

```
> set.seed(1) ; A <- crossprod(matrix(rnorm(1e4),1e2))
> x1 <- rep(1,100)
> x2 <- rep(1,100)
> microbenchmark::microbenchmark(x1 <- as.vector(arma_nipals(A,x1,100)) ,
+                                eigen_nipals(A,x2,100) )
Unit: microseconds
                                    expr     min      lq     mean  median      uq      max
 x1 <- as.vector(arma_nipals(A, x1, 100)) 576.456 576.996 589.8762 577.270 578.093 1429.413
                 eigen_nipals(A, x2, 100) 204.532 205.771 216.4862 206.358 206.784  996.272
 neval
   100
   100
```

```
> # is that the first eigenvector ? (yes)
> plot( data.frame( vec = eigen(A)$vector[,1], x1, x2 ) )
```



10.6 Conclusion ?

`RcppArmadillo` est globalement plus facile d'utilisation (tant du point de vue de la syntaxe que de la lecture de la documentation). L'avantage que nous constatons pour `RcppEigen` peut donner envie d'y investir.

Attention toutefois : les comparaisons peuvent varier. Armadillo (comme R) dépend des librairies BLAS et LAPACK. En installant une librairie BLAS optimisée (comme OpenBLAS) on peut espérer améliorer les performances de R et d'Armadillo – il faut recompiler R, voir manuel A.3.1 https://cran.r-project.org/doc/manuals/R-admin.html#Linear-algebra

Chapitre 11

Exemple : la régression logistique

11.1 Modèle et algorithme

TBD

11.2 Code Armadillo

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;
// [[Rcpp::export]]
List arma_logistic_model(arma::vec & y, arma::mat & x, double eps = 1e-5, int max_iter = 10) {
  int n(y.n_rows), p(x.n_cols);
  arma::vec W(n), pi(n), U(p), beta(p);
  arma::mat XWX(p,p), WX(n,p);

  beta.zeros();
  int k = 1;
  double U_norm = 0;

  do {
    for(int j = 0; j < n; j++) {
      pi(j) = 1/( 1 + exp( -dot(x.row(j), beta) ) );
      W(j) = pi(j)*(1-pi(j));
    }
    U = x.t() *(y - pi);

    WX = diagmat(W) * x;
    XWX = x.t() * WX;

    beta += solve(XWX, U);
    U_norm = norm(U);
    if(k++ > max_iter) break;
  } while (U_norm > eps) ;

  List L;
```

```
  L["beta"] = wrap(beta);
  L["cov.beta"] = wrap( inv_sympd(XWX) );
  return L;
}
```

11.3 Code Eigen

```
// [[Rcpp::depends(RcppEigen)]]
#include <RcppEigen.h>
using namespace Rcpp;
using namespace Eigen;
typedef Eigen::Map<Eigen::VectorXd> VEC;
typedef Eigen::Map<Eigen::MatrixXd> MAT;

// [[Rcpp::export]]
List eigen_logistic_model(NumericVector y_, NumericMatrix x_, double eps = 1e-5, int max_iter = 10) {
  VEC y(as<VEC>(y_));
  MAT x(as<MAT>(x_));

  int n(y.rows()), p(x.cols());
  VectorXd W(n), pi(n), U(p), beta(p);
  MatrixXd XWX(p,p), WX(n,p);

  beta.setZero();
  int k = 1;
  double U_norm = 0;

  do {
    for(int j = 0; j < n; j++) {
      pi(j) = 1/( 1 + exp( - x.row(j).dot(beta) ) );
      W(j) = pi(j)*(1-pi(j));
    }
    U.noalias() = x.transpose()*(y - pi);

    WX.noalias() = W.asDiagonal() * x;
    XWX.noalias() = x.transpose() * WX;

    beta += XWX.llt().solve(U);
    U_norm = U.norm();
    if(k++ > max_iter) break;
  } while (U_norm > eps) ;

  List L;
  L["beta"] = wrap(beta);
  L["cov.beta"] = wrap(XWX.llt().solve( MatrixXd::Identity(p,p) ));
  return L;
}
```

11.4 Benchmarking

```
> N <- 100
> expit <- function(x) 1/(1+exp(-x))
> set.seed(1); X <- cbind(1, rnorm(N), runif(N, -1, 1))
> pr <- expit( X %*% c(0.2, 0.1, -0.3) )
> Y <- rbinom(N, 1, pr)
> summary(glm( Y ~ X - 1, family=binomial()))

Call:
glm(formula = Y ~ X - 1, family = binomial())

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-1.6845  -1.2656   0.8278   0.9939   1.4375

Coefficients:
   Estimate Std. Error z value Pr(>|z|)
X1  0.28230    0.21418   1.318    0.187
X2 -0.08124    0.23461  -0.346    0.729
X3 -0.81584    0.38036  -2.145    0.032 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 138.63  on 100  degrees of freedom
Residual deviance: 130.32  on  97  degrees of freedom
AIC: 136.32

Number of Fisher Scoring iterations: 4
```

```
> L <- eigen_logistic_model(Y, X, 1e-5, 10)
> cbind(beta = L$beta, sd.beta = sqrt(diag(L$cov.beta)))
            beta   sd.beta
[1,]  0.28229951 0.2141818
[2,] -0.08123554 0.2346101
[3,] -0.81583666 0.3803551
```

```
> mbm <- microbenchmark::microbenchmark( eigen_logistic_model(Y,X), arma_logistic_model(Y,X))
> mbm
Unit: microseconds
                       expr    min      lq     mean  median      uq     max neval
 eigen_logistic_model(Y, X) 19.175 19.6660 30.07890 20.5575 20.9090 965.194   100
  arma_logistic_model(Y, X) 29.110 29.5955 40.59437 30.1830 30.5495 747.236   100
```

Chapitre 12

Introduction à la librairie standard

12.1 Vecteurs

Les vecteurs `std::vector` peuvent grandir grâce à `push_back`, et rétrécir avec `pop_back`. On peut aussi modifier leur taille avec `resize`. L'exemple suivant déclare un vecteur de longueur 4, y ajoute un élément égal à 5.25, l'enlève, et redimensionne le vecteur à la taille de 8 éléments.

```
#include <Rcpp.h>
#include <vector>
using namespace Rcpp;
//[[Rcpp::export]]
void demo() {
  std::vector<double> x(4);
  for(auto a : x) Rcout << a << " "; Rcout << "\n";
  x.push_back(5.25);
  for(auto a : x) Rcout << a << " "; Rcout << "\n";
  x.pop_back();
  for(auto a : x) Rcout << a << " "; Rcout << "\n";
  x.resize(8);
  for(auto a : x) Rcout << a << " "; Rcout << "\n";
}
```

Note : j'ai mis `#include <vector>` pour le principe. En fait l'inclusion de `Rcpp.h` suffit, car elle implique l'inclusion de nombreuses composantes de la librairie standard, dont celle-ci.

On peut anticiper sur les possibilités de changements de taille avec `reserve`.

```
#include <Rcpp.h>
using namespace Rcpp;

//[[Rcpp::export]]
NumericVector extrait_non_NAs(NumericVector x) {
  std::vector<double> R; // vecteur de doubles, vide par défaut
  R.reserve( x.size() ); // on prévoit de pouvoir étendre la taille de R jusqu'à x.size
  for(auto a : x) {
    if(!NumericVector::is_na(a))
      R.push_back(a);
  }
  return wrap(R);
}
```

12.2 Quelques algorithmes

```
#include <Rcpp.h>
using namespace Rcpp;

//[[Rcpp::export]]
NumericVector trie1(std::vector<double> x) {
  std::sort(x.begin(), x.end());
  return wrap(x);
}
//[[Rcpp::export]]
NumericVector trie2(std::vector<double> x, int n) {
  std::sort(x.begin(), x.begin() + n);
  return wrap(x);
}
//[[Rcpp::export]]
void trie3(NumericVector x) {
  std::sort(x.begin(), x.end());
}
```

TODO Ajouter au moins `std::fill` `std::accumulate` ...

12.3 Pairs & tuples

Définir une paire :

```
#include <Rcpp.h>
using namespace Rcpp;

//[[Rcpp::export]]
void exemple_de_paire() {
  std::pair<double, double> x;
  x.first = 10;
  x.second = 11.25;
  Rcpp::Rcout << "x.first = " << x.first << "\n";
  Rcpp::Rcout << "x.second = " << x.second << "\n";
}
```

On peut bien sûr faire des vecteurs de paires. Pour ne pas alourdir la syntaxe utilisons `typedef` pour définir `pts` comme abréviation de `std::pair<double, double>`:

```
#include <Rcpp.h>
using namespace Rcpp;

typedef std::pair<double, double> pts;

//[[Rcpp::export]]
void pts_vector() {
  std::vector<pts> x;
  for(int i = 0; i < 10; i++) {
    x.push_back( std::make_pair(R::rnorm(0,1), R::rnorm(0,1)) );
  }
  std::sort(x.begin(), x.end());
```

```
  // afficher contenu
  for(auto & a : x) {
    Rcout << "(" << a.first << "," << a.second << ")\n";
  }
}
```

12.4 tuples

```
#include <Rcpp.h>
using namespace Rcpp;

//[[Rcpp::export]]
void exemple_de_tuple() {
  std::tuple<double, double, int> x;
  x = std::make_tuple(0.25, 0.8, 1);
  Rcout << "(" << std::get<0>(x) << "," << std::get<1>(x)
        << "," << std::get<2>(x) << ")\n";
}
```

12.5 Maps

12.6 Heap

Chapitre 13

Programmation objet

13.1 Classes

13.1.1 Premiers exemples

On définit une classe `lapin` qui a deux membres (publics), `a` et `b`. Ceci permet de définie des objets de classe `lapin` ; si `bugs` est un tel objet, on accède à ses membres avec la syntaxe `bugs.a`, `bugs.b`.

La syntaxe des *constructeurs* permet d'appeler, dans la partie comprise entre les `:` et l'accolade ouvrante `{`, les constructeurs des différents membres de la classe, avant le corps de la fonction proprement dit. Le constructeur défini dans la classe ci-dessous, `lapin(int x, int y)`, appelle les constructeurs des entiers, se contentant de recopier la valeur de `x` dans `a`, et celle de `y` dans `b`. Comme il n'y a rien d'autre à faire, le corps de la fonction est vide, on referme immédiatement les accolades : `{ }`.

```
#include <Rcpp.h>
using namespace Rcpp;

class lapin {
private:
  int omega;
public:
  int a;
  int b;
  // constructeur
  lapin(int x, int y) : a(x), b(y) { }
}; // NE PAS OUBLIER CE POINT-VIRGULE

//[[Rcpp::export]]
void test_lapin() {
  lapin bugs(100,200);
  Rcpp::Rcout << "bugs.a = " << bugs.a << "\n";
  Rcpp::Rcout << "bugs.b = " << bugs.b << "\n";

  bugs.a++;
  Rcpp::Rcout << "bugs.a = " << bugs.a << "\n";
}
```

```
> test_lapin()
bugs.a = 100
```

```
bugs.b = 200
bugs.a = 101
```

On peut « perfectionner » cette classe en y ajoutant un membre privé `omega` et une fonction publique qui permet de lire la valeur de `omega` (mais pas de la modifier !).

Le constructeur initialise `a` et `b` dans la partie qui précède le corps de la fonction (entre accolades) ; puis la valeur `a + b` est copiée dans `omega` (corps de la fonction). On aurait pu aussi le déclarer comme `lapin(int x, int y) : omega(x+y), a(x), b(y) {}` (attention l'initialisation doit se faire dans l'ordre des membres de la classe, ici `omega` puis `a` et `b`).

```
#include <Rcpp.h>
class lapin {
private:
  int omega;
public:
  int a;
  int b;
  lapin(int x, int y) : a(x), b(y) {
    omega = a + b;
  }
  int get_omega() {
    return omega;
  }
};

//[[Rcpp::export]]
void essaye_lapin() {
  lapin bugs(100,200);
  Rcpp::Rcout << "bugs.a = " << bugs.a << "\n";
  Rcpp::Rcout << "bugs.b = " << bugs.b << "\n";
  // ceci déclencherait une erreur à la compilation :
  // Rcpp::Rcout << "bugs.omega = " << bugs.omega << "\n";

  Rcpp::Rcout << "bugs.get_omega() = " << bugs.get_omega() << "\n";
}
```

13.1.2 Un exemple plus complexe

Le membre `t` de cette classe contient un temps en secondes. Il est d'emblée déclaré privé ; on a également défini deux constructeurs et deux fonctions membre.

```
#include <Rcpp.h>
#include <iostream>
using namespace Rcpp;

class heure {
private:
  unsigned int t;

  std::tuple<int, int, int> hms() {
    int h = t/3600;
    int m = (t - 3600*h)/60;
    int s = t - 3600*h - 60*m;
```

```
    return std::make_tuple(h,m,s);
  }

public:
  // deux constructeurs
  heure(int h, int m, int s) : t( s + 60*(m + 60*h) ) { }
  heure() : t(0) { }

  // fonctions membres de la classe
  void step(int s = 1) {
    t += s;
  }

  void print() {
    auto x = hms();
    Rcpp::Rcout << std::get<0>(x) << ":" << std::get<1>(x) << ":" << std::get<2>(x) << "\n";
  }
};

// [[Rcpp::export]]
void heures() {
  heure a;
  a.print();
  a.step(70);
  a.print();
  heure b(13,25,59);
  b.print();
  b.step();
  b.print();
  heure c = b; // la copie est définie "par défaut"
  c.print();
}
```

```
> heures()
0:0:0
0:1:10
13:25:59
13:26:0
13:26:0
```

Note : pour indiquer au compilateur que la fonction `print` ne modifie pas la classe, on aurait pu la déclarer `const`, ainsi :

```
  void print() const {
    auto x = hms();
    Rcpp::Rcout << std::get<0>(x) << ":" << std::get<1>(x) << ":" << std::get<2>(x) << "\n";
  }
```

13.1.3 Déclaration séparée de la définition des fonctions

On peut déclarer la classe dans un fichier `heure.h` :

```
#ifndef HEURE_
#define HEURE
```

```
class heure {
  private:
  unsigned int t;

  std::tuple<int, int, int> hms();

  public:
  heure(int h, int m, int s);
  heure();

  void step(int s = 1);
  void print();
};
#endif
```

et définir l'implémentation dans un autre fichier, souvent `heure.cpp` :

```
#include <Rcpp.h>
#include "heure.h"

std::tuple<int, int, int> heure::hms() {
  int h = t/3600;
  int m = (t - 3600*h)/60;
  int s = t - 3600*h - 60*m;
  return std::make_tuple(h,m,s);
}

heure::heure(int h, int m, int s) : t( s + 60*(m + 60*h) ) { }

heure::heure() : t(0) { }

void heure::step(int s) {
  t += s;
}

void heure::print() {
  auto x = hms();
  Rcpp::Rcout << std::get<0>(x) << ":" << std::get<1>(x) << ":" << std::get<2>(x) << "\n";
}
```

La compilation avec plusieurs fichiers sera plus facile à mettre en œuvre lors de la création d'un package.

13.1.4 Surcharge d'opérateurs et autre sucre syntaxique

Une saine lecture : https://en.cppreference.com/w/cpp/language/operators

Voici une amélioration de la classe heure, où on définit un membre de la classe, l'opérateur `+=`, une fonction extérieure, l'opérateur `+`, et un opérateur `<<` qui permet d'utiliser la syntaxe `Rcpp::Rcout << x` où `x` est une heure.

Le fichier `heure.h` :

```
#include <iostream>
#ifndef HEURE_
#define HEURE
```

```
class heure {
private:
  unsigned int t;
  std::tuple<int, int, int> hms();

public:
  heure(int h, int m, int s);
  heure();

  void step(int s = 1);

  // les fonctions amies ne sont pas membre de la classe mais elles
  // accèdent à la partie "private"
  friend std::ostream& operator<<(std::ostream& ostr, heure h);

  // l'opérateur += *doit* renvoyer une référence (à l'objet lui-même)
  heure & operator+=(const heure & a);
};

// c'est une bonne pratique de définir l'opérateur + comme extérieur
heure operator+(heure a, const heure & b);
#endif HEURE
```

Le fichier heure.cpp :

```
#include <Rcpp.h>
#include "heure.h"

std::tuple<int, int, int> heure::hms() {
  int h = t/3600;
  int m = (t - 3600*h)/60;
  int s = t - 3600*h - 60*m;
  return std::make_tuple(h,m,s);
}

heure::heure(int h, int m, int s) : t( s + 60*(m + 60*h) ) { }

heure::heure() : t(0) { }

void heure::step(int s) {
  t += s;
}

// définition de l'opérateur +=
heure & heure::operator+=(const heure & a) {
  t += a.t;
  // dans toutes les fonctions membres de la classe, 'this' est un
  // pointeur vers l'objet lui même
  // ici il faut "se renvoyer soi-même" :
  return *this;
}

// définition de l'opérateur +
heure operator+(heure a, const heure & b) {
```

```
  a += b;
  return a;
}

// définition de l'opérateur <<
std::ostream& operator<<(std::ostream& ostr, heure h) {
  auto x = h.hms();
  ostr << std::get<0>(x) << ":" << std::get<1>(x) << ":" << std::get<2>(x);
  return ostr;
}
```

Définition de l'opérateur +=

Quand le compilateur rencontre une expression `x += a`, il appelle la fonction membre attachée à `x` (qui a donc accès directement à `x.t`, simplement via `t`) définie ci-dessus, avec argument `a` ; comme cette fonction est membre de la classe, elle a accès à l'élément privé `a.t`. Au lieu d'écrire `x += a` on pourrait écrire `x.operator+=(a)` (mais c'est moins joli).

La fonction que nous avons définie modifie la valeur de `x` à partir de la valeur de `a`. Elle doit ensuite, c'est une convention qui fait partie des règles d'utilisation des opérateurs, renvoyer une référence à `x`. Ceci est réalisé par la ligne `return *this;`.

Qu'est-ce que c'est que `this` ? C'est, dans toutes les fonctions membres de la classe, un pointeur vers l'objet courant. Donc, `*this` est l'objet lui-même. La référence est construire par le compilateur, du fait que le type de la fonction est déclaré `heure &`.

Définition de l'opérateur +

On a défini + à partir de +=. Ce n'est pas un membre de la classe, donc contrairement à +=, dont le premier argument était implicite (c'était `*this`) elle a deux arguments, `a` et `b`. Un choix judicieux dans la présence ou l'absence de référence fait que `a` est copié lors de l'appel de la fonction ; on utilise += pour y ajouter `b`, et on renvoie cette copie.

On aurait pu définir l'opérateur + comme un membre de la classe. On l'aurait alors déclaré comme une fonction à un seul argument, le premier membre de l'opération étant l'objet lui-même.

```
heure heure::operator+(const heure & a) {
  heure x;
  x.t = t + a.t;
  return x;
}
```

Il y a de bonnes raisons de préférer définir l'opérateur + comme extérieur à la classe comme on l'a fait. L'une d'elles est de limiter le nombre de fonctions membres, ayant donc accès à la partie privée ; si on décidait de changer le contenu de la partie privée (la façon dont on représente les heures), par exemple en les stockant sous la forme de trois entiers pour heures, minutes, secondes, il faudrait modifier l'implémentation de +=, mais il n'y aurait rien de plus à faire pour +.

Définition de l'opérateur <<

Les opérateurs « chevrons » sont des fonctions à deux variables : la syntaxe `a << b` correspond à un appel avec les arguments `a` et `b`. Dans l'utilisation qui en est faite pour afficher des choses à l'écran, le premier argument `a` est une référence à un `ostream`, pour « output stream » (c'est le cas de `Rcout`); la fonction renvoie cet argument, ce qui permet d'enchaîner les appels avec la syntaxe `a << b1 << b2`, etc.

TODO illustrer fonctionnement ...

13.2 Hérédité

Définissons une classe `rdv` qui hérite de façon publique de la classe `heure`.

Tous les objets de classe `rdv` sont également de classe `heure` ! Notez la façon dont on appelle

```
#include <Rcpp.h>
#include "heure.h"
class rdv : public heure {
  public:
  std::string lieu;
  rdv(std::string li, heure h) : heure(h), lieu(li) { }
  friend std::ostream& operator<<(std::ostream& ostr, rdv r);
};

std::ostream& operator<<(std::ostream& ostr, rdv r) {
  ostr << r.lieu << " à " << (heure) r;
  return ostr;
}

// [[Rcpp::export]]
void classe_derivee() {
  rdv X("Gare de l'est", heure(9,30,0));
  Rcpp::Rcout << X << "\n";

  // comme X est aussi un objet de la classe "heure",
  // la fonction step est définie :
  X.step(60);
  Rcpp::Rcout << X << "\n";

  // l'addition avec un autre objet de classe heure est définie également :
  X += heure(1,0,0);
  Rcpp::Rcout << X << "\n";
}
```

13.3 Fonctions virtuelle et classes abstraites

On peut définir des classes où certaines méthodes sont « virtuelles », c'est-à-dire qu'elles peuvent être redéfinies dans les classes dérivées. Une méthode peut également être purement virtuelle, auquel cas sa définition dans une classe dérivée est non seulement possible, mais nécessaire.

```
#include <Rcpp.h>
#include <Rcpp.h>

class FUN0 {
  private:
  double eps;
  public:
  virtual double f(double x) = 0; // fonction purement virtuelle
  virtual double df(double x) {
    return (f(x+eps) - f(x-eps))/(2*eps);
  }
  FUN0() : eps(0.001) {} ;
```

```
  FUN0(double e) : eps(e) {};
};

class poly : public FUN0 {
  std::vector<double> a;
  int n;
  public:
  poly(std::vector<double> a_) : a(a_), n(a.size()) {}
  poly(std::vector<double> a_, double e) : a(a_), n(a.size()), FUN0(e) {}
  double f(double x) {
    double R = 0;
    for(int i = n-1; i >=0 ; i--) {
      R = R*x + a[i];
    }
    return R;
  }
};

//[[Rcpp::export]]
void essai(double x) {
  std::vector<double> a {2, 1, -1};
  poly P(a);
  Rcpp::Rcout << P.f(x) << "\n";
  Rcpp::Rcout << P.df(x) << "\n";
}
```

13.4 Copie, Move, Destruction

Tous les objets ont un ou des constructeurs, un opérateur « move » (depuis C++11), et un destructeur. Nous n'avons déclaré que des constructeurs. Les aures opérateurs, en l'absence de déclaration de notre part, ont eu des valeurs par défaut. L'immense majorité du temps cela suffit.

Regardons quand-même à quoi correspondent ces opérations, en définissant deux classes plus ou moins bavardes.

```
#include <Rcpp.h>
using namespace Rcpp;

// Une classe très bavarde
class Alpha {
  public:
  unsigned int a;

  // Constructeur
  Alpha(int x) : a(x) {
    Rcpp::Rcout << "Alpha (" << a << ") a été construit\n";
  }

  // Construction par copie
  // Le const et le passage par référence sont obligatoires !
  Alpha(const Alpha & A) : a(A.a) {
    Rcpp::Rcout << "Alpha (" << a << ") a été construit par copie\n";
  }
```

```
  // Opérateur move. Le type 'Alpha &&' est une "référence à une rvalue"
  // Note : cet opérateur a le droit de modifier son argument
  // Le destructeur de l'argument X sera toujours appelé après un move
  Alpha(Alpha && X) : a(X.a) {
    X.a = 0; // par exemple
    Rcpp::Rcout << "Alpha (" << a << ") a bougé\n";
  }

  // Destructeur. Ce destructeur ne fait rien :
  // le compilateur se charge tout seul d'appeler le destructeur
  // des membres de la classe (ici, int a)
  ~Alpha() {
    Rcpp::Rcout << "Détruit Alpha (" << a << ")\n";
  }
};

// Une classe un peu moins bavarde
class Beta {
  private:
  Alpha a1;
  Alpha a2;

  public:
  Beta(int x) : a1(x), a2(x+1) {
    Rcpp::Rcout << "Beta (" << x << ") a été construit\n";
  }

  // Inutile d'appeler les destructeurs de a1 et a2
  // ils le seront de toute façon
  ~Beta() {
    Rcpp::Rcout << "Détruit Beta (" << a1.a << ")\n";
  }
};
```

On testera les exemples suivants. Création puis destruction quand on sort de la fonction (on vérifie que le destructeur des Alpha est bien appelé par celui des Beta) :

```
void creations() {
  Alpha x(7);
  Beta y(8);
}
```

Opérateur de copie de Alpha :

```
void copie_alpha() {
  Alpha x(7);
  Alpha y = x;
}
```

On n'a pas déclaré d'opérateur de copie pour Beta. Le compilateur l'a construit à partir de celui de Alpha :

```
void copie_beta() {
  Beta x(7);
  Beta y = x;
}
```

Que se passe-t-il dans une boucle ?

```
void en_boucle() {
  Rcpp::Rcout << "Création de Alphas dans une boucle\n";
  for(int i = 10; i < 12; i++) {
    Rcpp::Rcout << "** i = " << i << "\n";
    Alpha z(i);
  }
}
```

Et si on copie les éléments dans un vecteur ?

```
//[[Rcpp::export]]
void vecteur() {
  Rcpp::Rcout << "Création d'un vecteur de Alphas\n";
  std::vector<Alpha> V(4);
  V.reserve(4);  // essayer d'enlever ça pour voir ce qui se passe !!!
  Rcpp::Rcout << "Boucle\n";
  for(int i = 10; i < 12; i++) {
    Rcpp::Rcout << "** i = " << i << "\n";
    Alpha z(i);
    V.push_back(z);
  }
  Rcpp::Rcout << "\n** Fin de boucle ! **\n\n";
}
```

On comparera avec ce qui se passe quand on utilise `V.push_back(Alpha(i));` pour remplir `V` (entrée en scène de move). (Note : avant C++11 l'opérateur `swap` pouvait remplir le rôle joué par move ici).

En pratique : il est rare d'avoir à définir soi-même ces opérateurs, sauf si on décide d'allouer de la mémoire avec l'opérateur `new` ; il faut alors la libérer avec l'opérateur `delete` sous peine de graves ennuis. Est-ce bien raisonnable ? L'utilisation d'un `std::vector` est suffit la plupart du temps à nos besoins de stockage.

Chapitre 14

Exemple : Metropolis-Hastings (3)

14.1 Une classe fonctionnelle

La fonction $\Pi(x_1, x_2)$ que nous avons considérée fait partie de la famille de fonctions

$$\Pi(x_1, x_2; a, b, c, \alpha) = \left(1 + ax_1^2 + 2bx_1x_2 + cx_2^2\right)^\alpha$$

paramétrée par a, b, c et α. Pour qu'elle soit proportionnelle à une densité il y a quelques conditions à remplir, cela ne nous concerne pas. On peut facilement créer une classe munie d'un opérateur `()`.

```
#include <Rcpp.h>
using namespace Rcpp;

#ifndef PI_class
#define PI_class

class Pi_family {
  public:
    double a, b, c, alpha;
    Pi_family(double a_, double b_, double c_, double alpha_) :
      a(a_), b(b_), c(c_), alpha(alpha_) {};

    double operator()(double x1, double x2) {
      return pow( 1 + a*x1*x1 + 2*b*x1*x2 + c*x2*x2, alpha);
    }
};


#endif
```

14.2 Mise en œuvre

```
#include <Rcpp.h>
#include "metropolis.h"
#include "Pi.h"
```

```
using namespace Rcpp;

//[[Rcpp::export]]
NumericMatrix MH_cpp_3(int B, double sd, int burn = 0, int thin = 1) {
  Pi_family F(1.0, 0.5, 1.0, -3.0);
  return Metropolis(F, B, sd, burn, thin);
}
```

Chapitre 15

Multi-threading avec RcppParallel

Installer `RcppParallel`.

Avertissement : sauf cas très particuliers, le multithreading, et aucune autre technique de parallélisation des calculs, ne donne pas les résultats qu'on pourrait attendre naïvement, c'est-à-dire que si le temps d'exécution d'un calcul non parallélisé est de N secondes, alors quand on le parallélise sur K cœurs on fait le même calcule en $\frac{N}{K}$ secondes.

Il y a plusieurs raisons à cela. L'une d'elle est un surcoût lié à la création des processus qui vont exécuter les calcules ; une autre est que ces processus vont tenter de lire simultanément des données en mémoire, et vont donc se gêner mutuellement.

En pratique, on observe que le temps d'exécution décroît quand on augmente le nombre K de cœurs *jusqu'à une certaine valeur de* K, puis stagne, et remonte. Il peut donc être contre-productif, si vous disposez d'une machine avec 64 cœurs, de lancer votre calcul sur 64 cœurs ! Il aurait pu aboutir plus rapidement si vous aviez utilisé 8 cœurs seulement. Faites des expériences pour déterminer le nombre idéal de cœurs.

15.1 Classes `RVector` et `RMatrix`

L'intervention de R dans la gestion de la mémoire (ramasse-miettes ou garbage collector) empêche d'utiliser des objets `Rcpp` dans les threads. Il faut donc utiliser les objets de C++ ; ou bien, et cela permet d'éviter la copie, les objets `RVector` et `RMatrix` définis par `RcppParallel`.

Ces classes permettent de manipuler directement les éléments d'une matrice ou d'un vecteur sans que jamais une fonction interne de R ne soit appelée implicitement. Elles ont des fonctions membre globalement similaires à celles des classes de vecteurs et de matrices de R. Ce sont des classes templatées, il faut préciser si on déclare un `RVector<int>` ou un `RVector<double>`.

L'exemple ci-dessous illustre le fonctionnement de ces classes.

```
// [[Rcpp::depends(RcppParallel)]]
#include <RcppParallel.h>
#include <Rcpp.h>

using namespace Rcpp;
using namespace RcppParallel;

// [[Rcpp::export]]
void rvec(NumericVector x) {
  RVector<double> a(x);
```

```
  for(size_t i = 0; i < a.size(); i++)
    Rcout << a[i] << " ";
  Rcout << "\n";
}

// [[Rcpp::export]]
void rmat(NumericMatrix x) {
  RMatrix<double> a(x);
  for(size_t i = 0; i < a.nrow(); i++) {
    for(size_t j = 0; j < a.ncol(); j++) {
      Rcout << a(i,j) << " ";
    }
    Rcout << "\n";
  }
}
```

```
> a <- rnorm(3)
> a
[1]  0.8936737 -1.0472981  1.9713374
```

```
> rvec( a)
0.893674 -1.0473 1.97134
```

```
> b <- matrix(rnorm(9), 3)
> b
           [,1]        [,2]       [,3]
[1,] -0.3836321  0.08296573 0.32300650
[2,]  1.6541453  0.56722091 1.04361246
[3,]  1.5122127 -1.02454848 0.09907849
```

```
> rmat(b)
-0.383632 0.0829657 0.323007
1.65415 0.567221 1.04361
1.51221 -1.02455 0.0990785
```

15.2 Classe `Worker`

La classe `Worker` est une classe abstraite, dont l'opérateur `operator()(size_t begin, size_t int)` est purement virtuel. Il faudra donc définir des classes dérivées dans lesquels cet opérateur sera défini ; son rôle sera de faire un calcul, typiquement « pour `i` de `begin` à `end` ».

Les résultats des calculs pourront être agrégés de deux façons différentes, selon qu'on utilisera la fonction `parallelReduce` (cas typique : boucle qui somme les éléments d'un vecteur) ou `parallelFor` (cas typique : boucle qui remplit un vecteur).

Les exemples qui suivent illustrent ces deux cas.

15.3 Utilisation de `ParallelReduce`

```
// [[Rcpp::depends(RcppParallel)]]
#include <RcppParallel.h>
#include <Rcpp.h>
```

```
using namespace Rcpp;
using namespace RcppParallel;


class para_somme : public Worker {
public:
  // les données d'entrée
  const RVector<double> vec;

  // contiendra la somme
  double S;

  // constructeurs
  para_somme(const NumericVector vec) : vec(vec), S(0) {}
  para_somme(const para_somme & A, Split) : vec(A.vec), S(0) {}

  // l'opérateur () fait tout le boulot
  void operator()(size_t begin, size_t end) {
    for(size_t i = begin; i < end; i++)
      S += vec[i];
  }

  // la fonction membre join "recolle les morceaux"
  void join(const para_somme & Q) {
     S += Q.S;
  }
};

//[[Rcpp::export]]
double somme_parallele(NumericVector x) {
  para_somme A(x);
  parallelReduce(0, x.size(), A);
  return A.S;
}
```

```
> a <- rnorm(1e7)
> RcppParallel::setThreadOptions(1)
> microbenchmark::microbenchmark(sum(a), somme_parallele(a))
Unit: milliseconds
               expr      min       lq     mean   median       uq      max neval
             sum(a) 7.762418 7.801796 7.835239 7.810889 7.833459 8.898114   100
 somme_parallele(a) 7.726328 7.765234 7.807666 7.780655 7.815088 8.725157   100
```

```
> RcppParallel::setThreadOptions(2)
> microbenchmark::microbenchmark(sum(a), somme_parallele(a))
Unit: milliseconds
               expr      min       lq     mean   median       uq       max neval
             sum(a) 7.769733 7.874707 8.340467 7.991085 8.307337 15.486579   100
 somme_parallele(a) 4.129029 4.427790 4.692995 4.542516 4.762561  8.567485   100
```

15.4 Utilisation de `ParallelFor`

```
// [[Rcpp::depends(RcppParallel)]]
#include <Rcpp.h>
#include <RcppParallel.h>
using namespace Rcpp;
using namespace RcppParallel;

class zorglub : public Worker {
public:
   // Pointeur vers fonction à appliquer
   double (*F) (double);
   // point de départ
   double debut;
   // step
   double step;

   // resultat
   RVector<double> r;

   zorglub(double (*F_) (double), double debut_, double fin, NumericVector resultat) :
     F(F_), debut(debut_), step((fin-debut)/(resultat.size() -1)), r(resultat) {
     if(resultat.size() < 2)
       step = 0;
   }

   void operator()(size_t beg, size_t end) {
     double x = debut + beg*step;
     for(size_t i = beg; i < end; i++) {
       r[i] = F(x);
       x += step;
     }
   }
};

double f(double x) {
  return sin(x);
}

// [[Rcpp::export]]
NumericVector Zz(double a, double b, double n) {
  NumericVector resultat(n);
  zorglub A(f, a, b, resultat);
  parallelFor(0, n, A);
  return resultat;
}
```

```
> RcppParallel::setThreadOptions(1)
> microbenchmark::microbenchmark( Zz(0, 100, 1e6) )
Unit: milliseconds
              expr      min       lq   mean   median       uq      max neval
 Zz(0, 100, 1e+06) 15.51751 15.98331 17.317 17.28737 17.62263 26.03253   100
```

```
> RcppParallel::setThreadOptions(2)
> microbenchmark::microbenchmark( Zz(0, 100, 1e6) )
Unit: milliseconds
              expr      min       lq     mean   median       uq      max neval
 Zz(0, 100, 1e+06) 8.517078 8.680065 9.967616 10.20032 10.25074 17.32961   100
```

```
> RcppParallel::setThreadOptions(4)
> microbenchmark::microbenchmark( Zz(0, 100, 1e6) )
Unit: milliseconds
              expr     min      lq     mean   median       uq      max neval
 Zz(0, 100, 1e+06) 4.75617 4.83022 6.231552 6.444985 6.492114 13.31284   100
```

15.5 Un exemple contre-productif avec `ParallelFor`

```
// [[Rcpp::depends(RcppParallel)]]
#include <Rcpp.h>
#include <RcppParallel.h>
using namespace Rcpp;
using namespace RcppParallel;

class para_vsomme : public Worker {
public:
   const RVector<double> a, b;
   RVector<double> s;

   para_vsomme(const NumericVector a_, NumericVector b_, NumericVector s_)
      : a(a_), b(b_), s(s_) { }

   void operator()(size_t beg, size_t end) {
     for(size_t i = beg; i < end; i++)
       s[i] = a[i] + b[i];
   }
};

// [[Rcpp::export]]
NumericVector vsomme_parallele(NumericVector x, NumericVector y) {
  NumericVector resultat(x.size());
  para_vsomme A(x, y, resultat);
  parallelFor(0, x.size(), A);
  return resultat;
}
```

```
> a <- rnorm(1e7)
> b <- rnorm(1e7)
> RcppParallel::setThreadOptions(1)
> microbenchmark::microbenchmark(a+b, vsomme_parallele(a, b))
Unit: milliseconds
                  expr      min       lq     mean   median      uq       max neval
                 a + b 27.68184 28.07352 33.59273 35.66337 38.1583  42.01913   100
 vsomme_parallele(a, b) 35.59806 36.01048 42.78391 43.19031 45.4893 144.13705   100
```

```
> RcppParallel::setThreadOptions(2)
> microbenchmark::microbenchmark(a+b, vsomme_parallele(a, b))
Unit: milliseconds
                   expr      min       lq     mean   median       uq      max neval
                  a + b 27.66763 27.96717 34.60662 37.30133 38.20650 135.8419   100
 vsomme_parallele(a, b) 34.45590 34.83041 39.82405 37.72899 44.63973  71.2574   100
```

Chapitre 16

Créer un package